Succede che studente capisca la soluzione, ma dica di aver avuto problemi a risolverlo, potete caricare la vostra soluzione in maniera anonima, caricando semplicemente la foto della soluzione su questo link Dropbox
https://www.dropbox.com/request/C54bYCgXuAbMjKBWjiuf
in modo che io possa capire qual è il problema che avete incontrato, se esistono problemi comuni a più studenti sarà interessante spiegare meglio in aula tale problematiche.
Potete farlo sia per esercizi risolti a casa che quelli a lezione.

Esercizio 1)

We have stressed the need for an operating system to make efficient use of the computing hardware. When is it appropriate for the operating system to forsake this principle and to "waste" resources? Why is such a system not really wasteful?

What is the main difficulty that a programmer must overcome in writing an operating system for a real-time environment?

Keeping in mind the various definitions of **operating system,** consider whether the operating system should include applications such as Web browsers and mail programs. Argue both that it should and that it should not, and support your answers.

How does the distinction between kernel mode and user mode function as a rudimentary form of protection (security) system?

Esercizio 2)

A CPU-scheduling algorithm determines an order for the execution of its scheduled processes. Given n processes to be scheduled on one processor, how many different schedules are possible? Give a formula in terms of n.

Esercizio 3)

Implementare algoritmo del banchiere e determinare la sua complessità computazionale

Esercizio 4)

Si consideri un sistema contenente tre risorse A, B e C, rispettivamente con 10, 3 e 8 istanze. Si assuma che a un dato istante t siano in esecuzione i processi P_0, P_1, P_2 e P_3, e che le allocazioni correnti delle risorse ai processi e il numero massimo di richieste di allocazione siano descritte dalle seguenti matrici.

	Assegnate			Massimo		
	A	B	C	A	B	C
P_0	3	1	2	7	2	4
P_1	2	0	1	2	1	3
P_2	0	2	1	5	2	5
P_3	1	0	1	3	0	2

Applicando l'algoritmo del banchiere si risponda alle seguenti domande:

1. La sequenza di processi $< P_0, P_1, P_2, P_3 >$ è sicura?
2. La sequenza di processi $< P_0, P_2, P_1, P_3 >$ è sicura?
3. La sequenza di processi $< P_3, P_2, P_0, P_1 >$ è sicura?
4. La sequenza di processi $< P_3, P_1, P_2, P_0 >$ è sicura?
5. Nell'istante t il sistema si trova in uno stato sicuro?

Se la risposta all'ultima domanda è affermativa, indicare quali delle seguenti richieste di risorse (ognuna da considerare partendo dallo stato all'istante t, indicando con R_i una richiesta fatta dal processo P_i) possono essere soddisfatte, sempre applicando l'algoritmo del banchiere:

6. $R_1 = (0, 0, 1)$,
7. $R_1 = (2, 0, 0)$,
8. $R_1 = (0, 0, 2)$.

Per prima cosa calcolate la matrice delle necessità, che rappresenta tutte le risorse che servono ancora al processo prima di poter essere eseguito. Si calcola semplicemente confrontando le risorse totali necessarie per il processo con quelle già assegnate.

Affinché le richieste possano essere soddisfatte, bisogna prima controllare quante risorse sono disponibili. Inoltre, ricordate che un processo non può richiedere più risorse di quelle che ha dichiarato come massimo. Perciò, se le risorse richieste, sommate a quelle già assegnate, superano il massimo dichiarato per quel processo, allora la richiesta non può essere accettata.

Esercizio 5)

Si consideri un sistema contenente quattro risorse A, B, C e D, rispettivamente con 8, 2, 7 e 2 istanze, allocate a un dato istante t ai processi $P_0, \ldots, P_4$ nel modo descritto dalle seguenti matrici.

	Assegnate					Massimo			
	A	B	C	D		A	B	C	D
P_0	1	0	1	1	P_0	1	1	1	2
P_1	2	0	0	0	P_1	3	1	0	1
P_2	1	1	1	0	P_2	1	1	3	1
P_3	0	1	2	0	P_3	1	2	2	0
P_4	3	0	1	1	P_4	4	0	3	1

Indicando con $R_i = (a, b, c, d)$ una richiesta rispettivamente di a istanze della risorsa A, b istanze della risorsa B e via discorrendo, fatta dal processo P_i, si considerino le seguenti richieste:

1. $R_0 = (1, 0, 1, 0)$,
2. $R_1 = (1, 0, 0, 0)$,
3. $R_2 = (0, 0, 1, 0)$,
4. $R_3 = (1, 1, 0, 0)$,
5. $R_2 = (0, 0, 2, 0)$.

Prima di controllare se le risorse possono essere assegnate, dovete verificare se esiste uno stato sicuro, cioè se esiste una sequenza sicura di esecuzione dei processi.

Applicando l'algoritmo del banchiere, determinare se le richieste siano o meno soddisfacibili, giustificando la risposta. Ciascuna richiesta deve essere considerata in modo indipendente dalle altre. Alla luce delle risposte date al punto precedente, il sistema al tempo t si trovava in uno stato sicuro?

Esercizio 6)

Si considerino i seguenti processi:

```
Risorse condivise
 semaphore  S, T, U;
 int x=100;

Processo P1
{
  wait(S);
  if (x=0)
    then
       signal(T)
    else
       signal(U)
  endif;
  wait(S);
  write(x);
}

Processo P2
{
   wait(T);
   x:=x−1;
   signal(S);
}

Processo P3
{
   wait(U);
   x:=x+1;
   signal(S);
}
```

Si supponga che i processi vengano eseguiti concorrentemente sulla stessa CPU. Quale è l'output di tale programma se i semafori sono inizializzati come segue S=1, T=0, U=0? Supponendo invece di inizializzare S=0, T=1, U=1, si può incorrere in una race condition?

Esercizio 7)

Si consideri il seguente programma concorrente:

Variabili condivise semaphor s1:=0, s2:=0;	

process **P1** {
signal(s1);
wait(s2);
print "P"
wait(s2);
print "R"
signal(s1);
}
	process **P2** {
	signal(s2);
	wait(s1);
	print "A";
	signal(s2);
	wait(s1);
	print "I";
	}

Descrivere il comportamento e i possibili output dell'esecuzione parallela di **P1** e **P2**.

Esercizio 8)

Si considerino i seguenti processi eseguiti correntemente sulla stessa CPU:

```
semaphore M, S, T, U;
int x=0;
```

Processo P1	**Processo P2**	**Processo P3**

`while(1)`
`{`
`wait(S);`
`wait(M);`
`x=x+1;`
`signal(M);`
`signal(T);`
`}`
	`while(1)`
	`{`
	`wait(T);`
	`wait(M);`
	`write(x);`
	`signal(M);`
	`signal(S);`
	`}`
		`wait(U);`
		`while(1)`
		`{`
		`wait(M)`
		`x=0;`
		`signal(M)`
		`}`

Qual è l'output di tale programma se i semafori sono inizializzati come segue:

1. `M=1 S=1 T=0 U=0`
2. `M=1 S=0 T=1 U=0`
3. `M=1 S=1 T=0 U=1`

Qual è il ruolo del semaforo `M` in tale programma?

Esercizio 9)

Si consideri il problema della sincronizzazione degli arrivi/partenze dei nuotatori presso una piscina. Siano presenti due classi di risorse, entrambe in numero limitato: n spogliatoi e k cesti in cui depositare i propri abiti (generalmente $n<k$). La procedura a cui ciascun utente dell'impianto sportivo deve sottostare è la seguente:

1) Trovare uno spogliatoio libero
2) Reperire un cesto per la biancheria
3) Riporvi i propri abiti
4) Lasciare lo spogliatoio e depositare il cesto al personale addetto
5) Nuotare (si suppone che la piscina abbia ricettività illimitata)
6) Recuperare il proprio cesto della biancheria
7) Trovare uno spogliatoio libero
8) Rivestirsi

Descrivere una soluzione al suddetto problema che garantisca la mutua esclusione per l'accesso alle risorse. Verificare la presenza di stallo nella soluzione proposta ed evidenziare se e come possa avvenire.
Suggerimento: Utilizzare due semafori contatori, opportunamente inizializzati, per l'accesso alle risorse.

Esercizio 10)

Si consideri un sistema costituito da quattro processi, tre dei quali sono processi "attuatori", che eseguono un'azione – la stessa per tutti –, mentre il quarto è un "fornitore" di risorse. Per mettere in atto l'azione, ciascun processo necessita di tre risorse distinte, ma ognuno ne possiede effettivamente solo una. Il fornitore mette a disposizione due risorse (di cui ha disponibilità infinita) scelte a caso ed il processo che possiede la risorsa del terzo tipo può portare a termine l'azione. Sincronizzare, mediante semafori, l'esecuzione dei quattro processi.

Attuatore_1 necessità risorsa_2 e risorsa_3 e possiede risorsa_1
Attuatore_2 necessità risorsa_1 e risorsa_3 e possiede risorsa_2
Attuatore_3 necessità risorsa_1 e risorsa_2 e possiede risorsa_3

Quando Attuatore_i ha tutte e 3 le risorse allora può essere eseguito

Esercizio 11)

Due processi *P* e *Q* comunicano tramite un buffer di capacità un messaggio. *P* invia messaggi che sono stringhe con dimensione massima di 80 caratteri. *Q* estrae i caratteri uno alla volta. Definire i dati condivisi e scrivere le procedure **invio**(x,y), con x array di 80 caratteri ed y dimensione di x, e **ricezione**(char ch). (**Suggerimento**: utilizzare i semafori e definire le procedure in base a uno pseudocodice a piacere).

Esercizio 12)

Supponiamo che nella Facoltà di Ingegneria vi sia un solo locale destinato ai servizi igienici. Il locale è condiviso da tutti gli studenti, uomini e donne, con la regola che, in ogni istante, l'intero locale può essere utilizzato esclusivamente da persone dello stesso sesso (ma ha capacità illimitata!). Utilizzando il costrutto semaforico, simulare un protocollo di accesso (ingresso/uscita) per sincronizzare i possibili utenti del locale dei servizi (non preoccuparsi di evitare la starvation).

Esercizio 13)

Si consideri un sistema nel quale sono definiti il semaforo *sem* e i processi P_1 (con priorità 1), P_2 (con priorità 1) e P_3 (con priorità 2). Lo scheduling avviene con una politica a priorità, che prevede il prerilascio e assegna il processore al processo pronto di priorità più elevata (a pari priorità applica la politica FIFO). La politica applicata al semaforo è la FIFO. Al tempo *t* il processo P_1 è in esecuzione e il processo P_2 è pronto; inoltre il semaforo *sem* ha valore 0 e la sua coda contiene P_3. Dopo il tempo *t* si verifica la seguente sequenza di eventi:

1) P_1 esegue *wait(sem)*;
2) il processo in esecuzione esegue *signal(sem)*;
3) il processo in esecuzione esegue *wait (sem)*;
4) il processo in esecuzione esegue *signal(sem)*.

Si specifichi quale processo è in esecuzione dopo ogni evento e come si modificano il valore e la coda del semaforo *sem* e la ready queue.

Esercizio 14)

Si consideri il problema dei lettori–scrittori, con riferimento alla soluzione proposta nel seguito:

Variabili condivise

```
semaphore mutex=1, db=1
int rc=0 ;
```

```
Process reader {
        while (1)
        {
                wait(mutex);
                rc=rc+1;
                if (rc==1) wait(db);    (-2-)
                signal(mutex);
                   \\ lettura del file (risorsa condivisa)
                wait(mutex);
                rc=rc−1;
                if (rc==0) signal(db);  (-3-)
                signal(mutex);
                           \\ usa i dati letti
        }
}
```

```
Process writer {
        while (1)
        {
          wait(db);              (-0-)
             \\ scrittura del file
          signal(db);            (-1-)
        }
}
```

Che cosa accade se:

1. si eliminano gli statement (-0-) e (-1-)
2. si elimina lo statement (-2-)
3. si elimina lo statement (-3-)

Esercizio 15)

Un gruppo di studenti è in coda davanti alla segreteria per ritirare un modulo di richiesta di borsa di studio. Inizialmente, su un tavolo, vi è una pila di 50 moduli. Quando uno studente raggiunge la testa della fila, controlla se ci sono ancora moduli disponibili. In caso affermativo, ne ritira uno e se ne va. Viceversa, lo studente suona un campanello per segnalare il problema al personale della segreteria. Un impiegato arriva e depone sul tavolo una nuova pila di 50 moduli. Utilizzando le seguenti variabili condivise:

semaphore fila = 1; campanello = 0; disp_moduli = 0;
int moduli = 50;

si scrivano le procedure che descrivono il comportamento dei due processi studente ed impiegato.

Esercizio 16)

Un conto bancario è condiviso tra più persone. Ciascuna può depositare o prelevare soldi dal conto, ma il saldo non deve mai diventare negativo. Se una persona accreditata vuole effettuare un prelievo e ci sono soldi a sufficienza, lo fa aggiornando il saldo, altrimenti resta in attesa che il saldo renda possibile il prelievo e, non appena c'è stato un deposito, avverte le eventuali altre persone in attesa e cerca di effettuare il prelievo. Quando una persona deposita, avverte un'altra eventuale persona in attesa che c'è stato un deposito.
Scrivere un monitor con le funzioni prelievo e deposito per risolvere il problema.

Esercizio 17)

Una tribù di N selvaggi mangia in comune da una pentola che può contenere fino ad M (con M<N) porzioni di stufato di missionario; quando un selvaggio ha fame controlla la pentola:

- se non ci sono porzioni, sveglia il cuoco ed attende che questo abbia riempito di nuovo la pentola;
- se la pentola contiene almeno un pasto, si mette in attesa di essere servito.

Il cuoco controlla inizialmente che ci siano delle porzioni:

- se ci sono si addormenta, altrimenti, cuoce M porzioni e le serve ai cannibali in attesa.

Si descrivano possibili soluzioni al problema che impieghino semafori o monitor (in quest'ultimo caso, si descrivano le procedure **riempi** e **serviti)**

Esercizio 18)

Al termine della finale del Campionato del Mondo di Calcio del 2006, vinta dalla Nazionale italiana contro quella francese, la polizia tedesca ha il compito di dirigere i tifosi alle stazioni ferroviarie, evitando che gruppi di nazioni differenti vengano a contatto. A tal fine, ogni volta che gruppi di tifosi devono attraversare una piazza, arrivando da strade diverse, il capo della polizia decide che:

- se la piazza è libera può essere attraversata da qualunque tifoso;
- se nella piazza c'è almeno un tifoso italiano, nessun tifoso francese può attraversare la piazza;
- se nella piazza c'è almeno un tifoso francese, nessun tifoso italiano può attraversare la piazza.

Si implementi una soluzione mediante semafori (in pseudo–codice) per i processi `italiano` e `francese`.
Suggerimento: utilizzare tre semafori per ottenere una semplice soluzione simmetrica.

Esercizio 19)

Giulio e Giovanni lavorano in un bar specializzato in panini e cocktail abbinati. Giulio prepara un panino mentre, in contemporanea, Giovanni realizza il cocktail più adatto. Quando entrambi hanno terminato il loro lavoro, servono sia il panino che la bevanda ad un cliente. Queste azioni si ripetono per tutto il giorno. Si noti che Giulio non può servire il panino finchè Giovanni non ha finito di preparare il cocktail e viceversa. Descrivere i due processi Giulio e Giovanni utilizzando i semafori.

I grafi di allocazione delle risorse verranno visualizzati usando dei nodi circolari e rettangolari per denotare rispettivamente processi e risorse; gli archi di richiesta saran-no diretti da un processo a una risorsa, mentre gli archi di assegnazione avranno verso opposto. Infine, gli archi di richiesta futura saranno tratteggiati. Possibile identificare una situazione di deadlock ricercando dei cicli nel grafo.

Esercizio 20)

Un sistema è composto da sette processi P1,. ..,P7 e da sei risorse R1, . . . , R6 ognuna avente un'unica istanza. A un dato istante, lo stato del sistema è il seguente:

- P1 sta utilizzando R1 e R2;
- P2 non sta utilizzando risorse ed è in attesa di R3;
- P3 non sta utilizzando risorse ed è in attesa di R2;
- P4 sta utilizzando R4 ed è in attesa di R2 e R3;
- P5 sta utilizzando R3 ed è in attesa di R5;
- P6 sta utilizzando R6 è ed è in attesa di R3;
- P7 sta utilizzando R5 ed è in attesa di R4.

Determinare se il sistema si trovi o meno in deadlock, per trovare cicli in grafo orientato ripassare «DAG e ordinamento topologico» del corso di DA1

Esercizio 21)

Per decidere se accettare una richiesta di accesso a una risorsa, si procede nel seguente modo:

a) Verificare la validità della richiesta, controllando che il processo abbia effettivamente richiesto (o possa richiedere) quella risorsa.
b) Controllare la disponibilità della risorsa. Se la risorsa è disponibile, è possibile considerare l'assegnazione.
c) Verificare che l'assegnazione non generi un ciclo nel sistema. Se l'assegnazione non provoca cicli, la richiesta può essere accettata.
In questo modo si garantisce che l'accesso alla risorsa sia coerente e non porti a situazioni di blocco nel sistema.

Un sistema è costituito da sette processi $P_1, \ldots, P_7$ e da sei risorse $R_1, \ldots, R_6$, ognuna avente un'unica istanza. A un dato istante t lo stato del sistema è il seguente:

- P_1 è in attesa di R_1;
- P_2 sta utilizzando R_3 e R_4;
- P_3 sta utilizzando R_1 ed è in attesa di R_3;
- P_4 non sta utilizzando risorse ed è in attesa di R_3;
- P_5 non sta utilizzando risorse ed è in attesa di R_4 e R_5;
- P_6 sta utilizzando R_2;
- P_7 sta utilizzando R_5 ed è in attesa di R_4;

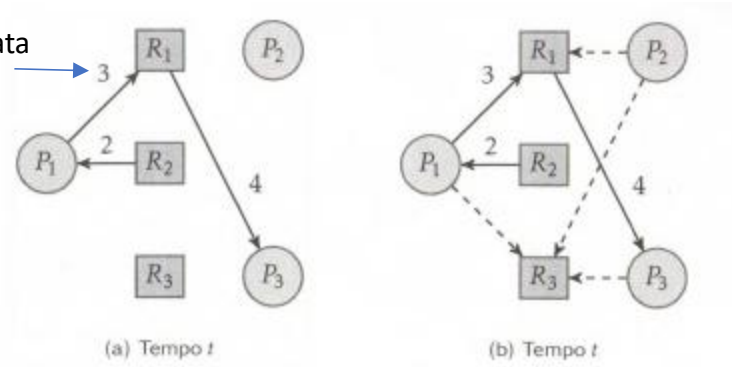
Inoltre, P_1 e P_2 hanno dichiarato che in futuro accederanno a R_6, così come P_5 ha dichiarato che accederà a R_3.

1. Si disegni il grafo di allocazione delle risorse, specificando gli archi di assegnazione, di richiesta e di richiesta futura, e lo si utilizzi per dimostrare che il sistema non si trova in stato di *deadlock*.
2. Come andrebbe gestita una richiesta di P_1 di accedere a R_6 all'istante t?
3. E una richiesta di P_5 di accedere a R_3 all'istante t?
4. E una richiesta di P_1 di accedere a R_2 all'istante t?
5. E una richiesta di P_2 di accedere a R_6 all'istante t?

Esercizio 22)

Partendo dal seguente stato iniziale

3 è la durata



Nel tempo iniziale si prevede che P1 utilizzi R2, anche se sta aspettando R1 per altre computazioni. Nello stesso modo P3 sta già usando R1. Algoritmo non considera che P3 stia usando R1 anche senza possedere R3, non abbiamo informazioni sul fatto che uso R1 possa finire senza arrivo di R3 (dunque ciclo creerebbe problemi), e.g. R1 puoi usarlo da solo fino ad un certo punto poi però ti serve R3 altrimenti non puoi finire.

Richiesta	Tempo di arrivo	Durata
$P_1 \rightarrow R_3$	$t+1$	2
$P_3 \rightarrow R_3$	$t+3$	3

Ipotizzando che queste vengano allocate applicando l'algoritmo basato sul grafo di allocazione, con l'accorgimento che nel momento in cui una qualsiasi risorsa viene liberata si proceda prima a provare a soddisfare delle richieste lasciate in sospeso e poi a convertire eventuali archi di richiesta in archi di assegnamento. In quale istante verranno soddisfatte tali richieste? E in quale istante le corrispondenti risorse verranno rilasciate?

Esercizio 23)

Supponiamo di ricevere 5 job (A,B,C,D,E) tali che:

Processo	Durata CPU–burst	Tempo di arrivo	Priorità
A	8	0	2
B	6	1	3
C	3	2	2
D	4	3	2
E	3	4	2

Descrivere la sequenza di esecuzione dei job (ad es. tramite Gantt chart) e calcolare l'istante in cui ogni job completa il suo task e il relativo tempo totale di completamento ottenuto con uno scheduling a **code multiple di priorità e feedback**, dove ogni coda è gestita con la **strategia FCFS**. Il **feedback** è definito come segue: la priorità diminuisce di 1 (fino al livello base 1) se si passano più di 6 unità di tempo (consecutive) in esecuzione nella CPU ed aumenta di 1 per ogni 6 unità di tempo passate in attesa in una qualsiasi coda.

FCFS si usa per la gestione delle code, non per effettuare preemption se nuovo processo aumenta priorità durante esecuzione di altro processo.

Esercizio 24)

Sei processi, P_1, P_2, P_3, P_4, P_5 e P_6 richiedono l'utilizzo contemporaneo della CPU. La lunghezza dei CPU–burst e le priorità sono dati nella seguente tabella (numeri bassi = priorità alte):

Processo	Durata CPU–burst	Priorità
P_1	6	1
P_2	4	2
P_3	8	3
P_4	2	4
P_5	4	5
P_6	6	6

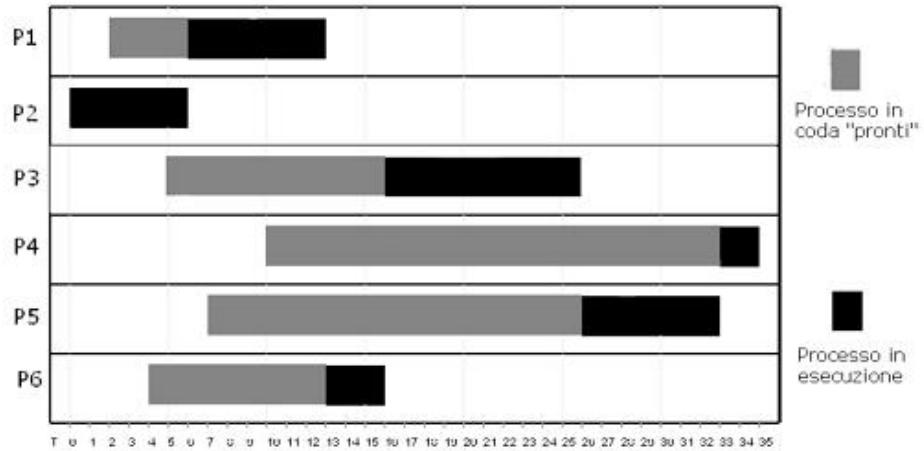
Si tracci il diagramma di Gantt relativo all'esecuzione dei sei processi e si calcoli il tempo di completamento di ogni processo per ciascuna delle seguenti politiche di scheduling della CPU (trascurando il tempo di context switch):

1. Shortest Job First;
2. Con Priorità;
3. Round Robin con quanto di tempo q = 2.

Esercizio 25)

Si considerino i processi P1, P2, P3, P4, P5 e P6 che vengono eseguiti, su un sistema monoprocessore, così come descritto in figura e si determini quale delle possibili politiche di scheduling sotto elencate può aver determinato la particolare sequenza di esecuzione:

- FIFO
- Round Robin (in caso di risposta positiva, indicare un esempio di ampiezza del quanto)



Giustificare la risposta fornita.

Esercizio 26)

Shortest Remaining Time First

Supponiamo di ricevere i seguenti processi:

Processo	A1	A2	A3	A4	B1	B2	B3	C1	C2	C3
Tempo di arrivo	0	2	8	11	2	4	9	3	5	7
Tempo di burst	4	2	1	4	5	4	3	7	8	2

Descrivere la loro sequenza di esecuzione tramite Gantt chart e calcolare l'istante in cui ogni job completa il suo task e il tempo medio di attesa ottenuto con uno scheduling a **code multiple di priorità** (con prelazione), dove, rispettivamente, la coda A è gestita con SRTF, la coda B con RR con quanto di tempo pari a 2msec e la coda C con FCFS.

Nota: il nome dei processi corrisponde alla coda in cui verranno ammessi al loro ingresso nel sistema.

Esercizio 27)

a) Si consideri la seguente coda di processi:

Processo	Lunghezza CPU burst (msec)
P_1	2
P_2	3
P_3	7
P_4	18

Ciascun processo esegue il proprio CPU burst, quindi richiede un'operazione di I/O della durata di 10 msec, esegue un altro CPU burst della stessa lunghezza del precedente, ed infine termina. Supponiamo inoltre che le operazioni di I/O possano procedere in parallelo. Si disegnino i diagrammi di Gantt relativi alle politiche di scheduling RR, con quanto di tempo pari a 2 msec, e SJF preemptive. In entrabi i casi, si valuti il tempo medio di attesa.

b)

Un insieme di task indipendenti, A, B, C e D, devono essere eseguiti su di un unico processore. I task possono venire elaborati più di una volta. Appena un task ha terminato la propria esecuzione, è pronto per la successiva (che non necessariamente durerà lo stesso tempo). I tempi di arrivo e di esecuzione sono descritti nella seguente tabella:

Task	Tempo di arrivo	1° esecuzione	2° esecuzione	3° esecuzione
A	0	10	6	4
B	2	3	2	2
C	3	2	–	–
D	5	1	1	–

Si tracci il diagramma di Gantt per lo scheduling FCFS e si calcoli il tempo di attesa relativo a ciascun processo. Lo scheduling generato soffre dell'*effetto convoglio*? Motivare la risposta data.

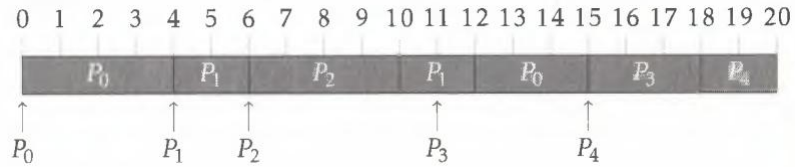
c)

Si consideri il seguente insieme di processi con relativi tempi di arrivo, tempi di esecuzione e di scadenza:

Processo	Tempo di arrivo (msec)	Esecuzione (msec)	Scadenza (msec)
T_1	0	10	33
T_2	4	3	28
T_3	5	10	29

Si tracci il diagramma di Gantt in base alla politica di scheduling EDF e si dica se tutte le scadenze risultano rispettate (lo scheduling è feasible, cioè ammissibile).

Esercizio 28)



illustra il diagramma di Gantt relativo allo scheduling di un insieme di processi, in cui ogni freccia indica il tempo di arrivo del corrispondente processo nella coda dei processi pronti. Analizzando tale diagramma, rispondere alle seguenti domande:

1. L'algoritmo utilizzato prelaziona la CPU ai processi?
2. Qual è il tempo massimo di turnaround? E qual è quello medio?
3. Considerando uno scheduling basato su priorità, qual è ordine di priorità dei 5 processi

Esercizio 29)

Si assuma a un dato istante t = 0 arrivino nella coda dei processi pronti, che precedentemente era vuota, i processi descritti nella seguente Tabella, ognuno indicato assieme alla relativa durata del prossimo CPU burst. Si disegni il diagramma di Gantt che descrive l'allocazione della CPU ai processi nel caso in cui lo scheduling venga effettuato in ordine di arrivo. Calcolare successivamente il tempo medio di turnaround, il tempo medio di attesa nella coda dei processi pronti e il numero medio del numero di processi in attesa nella stessa coda.

Processo	Durata
P_0	3
P_1	5
P_2	2
P_3	6
P_4	7
P_5	3

Come cambierebbero le risposte all'esercizio precedente se il sistema opera-tivo allocasse la CPU ai processi pronti utilizzando lo scheduling per brevità, assumendo che a parità di durata abbia priorità il processo P con il minore valore dell'indice i?

Esercizio 30)

Si consideri la Tabella che elenca i tempi di arrivo di un insieme di processi nella coda dei processi pronti e il relativo tempo di CPU burst. Assumendo che la CPU venga allocata ai processi utilizzando una poli-tica di scheduling per brevità (shortest job first) senza prelazione, si calcoli il tempo medio di turnaround dei processi. Mostrate il diagramma Gantt

Nome	Tempo di arrivo	Durata
P_0	0	3
P_1	8	8
P_2	11	1
P_3	15	1
P_4	17	5
P_5	19	6
P_6	20	6
P_7	21	1
P_8	23	1
P_9	28	1

Come cambierebbero le risposte all'esercizio precedente se il sistema opera-tivo prevedesse la prelazione?

Esercizio 31)

Sia dato un sistema operativo in cui la CPU venga assegnata ai processi pronti utilizzando una politica di scheduling circolare con time slice pari a 3 unità di tempo. Si considerino i processi descritti nella Tabella che per ognuno di essi riporta il tempo di arrivo nella coda dei processi pronti e il tempo di CPU burst.

1. Calcolare il tempo medio di turnaround dei processi, ipotizzando che quando due processi arrivano nel sistema nello stesso istante venga data priorità a quello con indice minore.
2. Rieseguire il punto precedente dell'esercizio calcolando il tempo medio di attesa in coda dei processi.
3. Rieseguire il punto precedente dell'esercizio calcolando il numero medio di processi nella coda dei processi pronti.

Nome	Tempo di arrivo	Durata
P_0	0	1
P_1	3	2
P_2	3	5
P_3	7	4
P_4	8	4
P_5	10	3
P_6	13	1
P_7	15	1
P_8	23	1
P_9	23	1

Esercizio 32)

An operating system supports a paged virtual memory, using a central processor with a cycle time of 1 microsecond. It costs an additional 1 microsecond to access a page other than the current one. Pages have 1000 words, and the paging device is a drum that rotates at 3000 revolutions per minute and transfers 1 million words per second. The following statistical measurements were obtained from the system:

- 1 percent of all instructions executed accessed a page other than the current page.
- Of the instructions that accessed another page, 80 percent accessed a page already in memory.
- When a new page was required, the replaced page was modified 50 percent of the time.

Calculate the effective instruction time on this system, assuming that the system is running one process only and that the processor is idle during drum transfers.

Tempo di accesso alla memoria secondaria:
10000 microsecondi per pagina

Dunque per trasferire
1000 words (i.e. one
page): 1000 microsecondi

Esercizio 33)

You have devised a new page-replacement algorithm that you think may be optimal. In some contorted test cases, Belady's anomaly occurs. Is the new algorithm optimal? Explain your answer.

Esercizio 34)

Si consideri un processo con 7 pagine logiche di 1K, e la seguente configurazione della tabella delle pagine (1=sì, 0=no):

Pag. Logica	Presente/Assente	Riferita	Modificata	Pag. Fisica
0	1	1	0	512
1	0	0	0	0
2	1	0	0	102
3	0	0	0	0
4	1	0	0	428
5	1	1	0	123
6	1	1	0	812

e della TLB (memoria associativa) di 4 posizioni

Posizione	Valida	Modificata	Pag. Virtuale	Pag. Fisica
0	1	0	5	123
1	1	1	6	812
2	0	0	0	0
3	1	0	0	512

Discutere cosa accade e mostrare come viene modificato il contenuto della TLB e della tabella delle pagine se il processo accede in scrittura alle locazioni 4000 e 5130.

Ogni pagina logica è 1K (detto nel testo), se ogni pagina logica è lunga 1000 allora la locazione 4000 cadrà nella pagina logica 3, la 5130 nella pagina 5. La pagina logica 0 contiene le locazioni 1-1000, la 1 le locazioni 1001-2000...

Esercizio 35)

Sia dato un sistema operativo con memoria centrale di 4 Kbyte, divisa in celle di un byte e allocata ai processi in modo contiguo, selezionando un'area libera tramite l'algoritmo *first fit* e allocando i processi nella sua parte iniziale, riducendo l'area libera alla memoria che avanza. Analogamente, quando un processo termina la memoria che questo occupava va a formare un'area libera, fondendo assieme eventuali aree libere contigue.

Processo	Grandezza	Tempo di esecuzione	Durata
P_1	2000	0	4
P_2	1010	2	7
P_3	500	3	5
P_4	580	6	10
P_5	1500	7	4

Visualizzare graficamente allocazione memoria

Esercizio 36)

Sia dato un sistema operativo con memoria centrale di 4 Kbyte, divisa in celle di un byte e allocata ai processi in modo contiguo, selezionando un'area libera tramite l'algoritmo *worst fit* e allocando i processi nella sua parte iniziale, riducendo l'area libera alla memoria che avanza.

Processo	Grandezza	Tempo di esecuzione	Durata
P_1	2048	0	10
P_2	1970	7	13
P_3	2000	11	19
P_4	30	13	10

Tecnica worst fit, assegna all'area libera concatenata più grande

Visualizzare graficamente allocazione memoria, inoltre fornire valutazione
Del grado di frammentazione esterna e interna all'istante t=13

Esercizio 37)

Si consideri un sistema operativo in cui la memoria centrale è allocata in modo distribuito ai processi tramite la tecnica della paginazione, utilizzando nel contampo tecniche di memoria virtuale. Si assuma che la memoria fisica sia divisa in *frame* di un Kbyte, che ogni indirizzo di memoria fisica occupi 32 bit e che in un indirizzo logico i bit più significativi indichino il numero di pagina e quelli meno significativi indichino lo scostamento.

Anche per questo punto presupponete di avere celle di 32 bit

1. Quanti bit sono necessari per specificare lo scostamento all'interno di una pagina?
2. Ipotizzando che anche l'indirizzo di una cella di memoria occupi 32 bit, qual è la quantità massima di memoria indirizzabile?

Si consideri la tabella delle pagine di processo, de-scritta nella Tabella in cui il bit di validità è impostato a 1 per le pagine già caricate in memoria centrale:

N. di pagina	bit di validità	N. di *frame*
0	1	739
1	1	127
2	0	–
3	1	91
4	0	–
5	1	470
6	1	2
7	1	106
8	1	495
9	0	–

Specificare che cosa succede quando il processo tenta di accedere ai seguenti indirizzi logici (espressi in notazione binaria, dividendo i bit in quartetti e abbreviando gli zero iniziali, al fine di facilitare la leggibilità), indicando ove possibile quali siano gli indirizzi fisici corrispondenti, esprimendoli in notazione decimale:

(a) $(0\ldots0\,0111\;1101\;0101)_2$,

(b) $(0\ldots0\,0100\;0001\;0111)_2$,

(c) $(0\ldots0\,1011\;1101\;1011)_2$.

Esercizio 38)

Si consideri un sistema operativo dotato di memoria virtuale con paginazione a richiesta pura. Si presupponga di eseguire un processo che richieda, nell'ordine, di accedere alle seguenti pagine:

2, 6, 3, 2, 9, 10, 9, 3, 11, 12, 3, 2.

Avendo a disposizione un massimo di 4 *frame* a cui associare le pagine del processo, si esegua l'algoritmo FIFO al fine di calcolare il numero di *page fault* che si verificano durante l'esecuzione del processo.

Esercizio 39)

Si consideri un sistema operativo dotato di memoria virtuale con paginazione a richiesta pura e rimpiazzamento con algoritmo LRU (*least recently used*). Si presupponga di eseguire un processo che richieda, nell'ordine, di accedere alle seguenti pagine:

8, 0, 10, 12, 9, 10, 11, 3, 11, 11.

Avendo a disposizione 3 *frame* a cui associare le pagine del processo, si calcoli il numero di *page fault* che si verificano durante l'esecuzione del processo.

Esercizio 40)

Si consideri un sistema operativo dotato di memoria virtuale con paginazione a richiesta pura e rimpiazzamento con algoritmo LRU (*least recently used*), abbinato a uno schema di prepaginazione che carica le prime pagine di un processo fino a riempire i *frame* disponibili. Si presupponga di eseguire un processo che richieda, nell'ordine, di accedere alle seguenti pagine:

15, 15, 0, 2, 3, 11, 2, 6, 4, 14.

Avendo a disposizione 4 *frame* $f_0, \ldots, f_3$ cui associare le pagine del processo, e ipotizzando che quando vi siano più *frame* in cui inserire una pagina questa venga assegnata al *frame* f_i con il minimo indice possibile, si calcoli il numero di *page fault* che si verificano durante l'esecuzione del processo.

Esercizio 41)

0:		1:	10	2:		3:		4:		5:	18
6:		7:		8:	11	9:	16	10:	20	11:	−1
12:	5	13:		14:		15:		16:	1	17:	8
18:	17	19:		20:	−1	21:		22:		23:	23

ogni riquadro rappresenta un blocco da 4 Kbyte, a sinistra del quale è riportato in grigio il corrispondente indirizzo, e in ogni blocco viene ri-servato un byte per gestire il puntatore al prossimo blocco. Il contenuto di ogni riquadro può essere:

- un numero non negativo n, a indicare che il blocco è occupato dai contenuti di un file che prosegue nel blocco numero
- il numero - 1 , quando il blocco contiene la parte finale di un file;
- nulla, nel caso in cui il blocco sia libero.

Ipotizzando che su questo disco sia memorizzato un intero file System che contiene due file, memorizzati rispettivamente a partire dai blocchi con indirizzo 9 e 12, si risponda alle seguenti domande:

1. La rappresentazione dei blocchi del disco contiene degli errori? Perché?
2. Assumendo che lo spazio libero sul disco sia gestito utilizzando un vettore di bit (*bitmap*) in cui a ogni blocco è associato un bit il cui valore è pari a 1 se il blocco risulta libero e pari a 0 altrimenti, quale sarà il contenuto della *bitmap*?
3. Ipotizzando che lo spazio libero venga allocato scandendo la *bitmap* da sinistra verso destra, in quali blocchi verrebbe memorizzato un nuovo file lungo 21547 byte?

La soluzioni prevedono che f0 sia: 5 9 19 4 11 7

Esercizio 42)

Sia dato un *file system* allocato tramite *File Allocation Table* (FAT) su un disco in cui ai contenuti dei file sono dedicati 20 blocchi di 4 KByte l'uno, ognuno accessibile specificando come indirizzo un numero che va da 0 a 19. Si ipotizzi che nel *file system* siano presenti i seguenti file: f_0, i cui contenuti sono inseriti, nell'ordine, nei blocchi 5, 19, 4, 11 e 7, e f_1, i cui contenuti sono inseriti, sempre nell'ordine, nei blocchi 18, 12, 13, 1, 6, 15 e 10.

1. Visualizzare i contenuti della FAT, assumendo che all'interno di questa struttura dati venga memorizzato il valore −1 in corrispondenza dei blocchi liberi e −2 per indicare l'ultimo blocco di un file.
2. Qual è la dimensione massima per un nuovo file, misurata in byte?

Esercizio 43)

Si hanno due dischi da 200 GB, con Transfer Rate pari a 200 MB/sec. Si possono configurare i due dischi in modalità RAID0 oppure RAID1. Determinare per ciascuna delle due modalità:

a) La capacità complessiva ottenuta

b) Si ha un miglioramento per la tolleranza ai guasti? Definite *p* la probabilità che accada guasto in un disco, come diventa probabilità guasto (i.e. perdere dati) del sistema RAID0 e RAID1?

Un array di dischi RAID è composto da 4 dischi da 500 GB, ciascuno con Transfer Rate di 120 MB/sec. L'array può essere configurato in modalità RAID0, RAID10. Determinare per ciascuna possibile configurazione:

- La capacità complessiva di memoria utilizzabile (Ctot)
- Il Transfer Rate (teorico) in lettura (TRL)
- Il Transfer Rate (teorico) in scrittura (TRS)

Considerate il caso del guasto prima di un disco e poi di un secondo disco: per ciascuna delle configurazioni RAID specificare se si ha perdita dei dati o no, oppure se ci sono rallentamenti in lettura e/o in scrittura.

Esercizio 44)

Si consideri un sistema nel quale sono definiti il semaforo sem e i thread P1 (con priorità 1), P2 (con priorità 1) e P3 (con priorità 2). Lo scheduling avviene con una politica a priorità, che prevede il prerilascio (preemption) e assegna il processore al processo pronto di priorità più elevata (a pari priorità applica la politica FIFO). La politica applicata al semaforo è la FIFO. Al tempo t il processo P1 è in esecuzione il processo P2 è pronto; inoltre il semaforo sem ha valore 0 e la sua coda contiene P3. Dopo il tempo t si verifica la seguente sequenza di eventi: P1 esegue P(sem)); il processo in esecuzione esegue V(sem); il processo in esecuzione esegue P(sem); il processo in esecuzione esegue V(sem). Si chiede di specificare quale processo è in esecuzione dopo ogni evento e inoltre come si modificano il valore e la coda del semaforo sem e la CodaPronti.

Esercizio 45)

In un sistema operativo, i thread A (produttore) e B (consumatore) cooperano scambiandosi messaggi attraverso un buffer di 10 celle, ciascuna capace di contenere un messaggio e utilizzando i semafori mutex (valore iniziale 1), CelleLibere (valore iniziale 10) e MessaggiGiacenti (valore iniziale 0). A meno delle operazioni di sincronizzazione e di quelle per la mutua esclusione (necessarie perché si suppone che le operazioni insert(item) e remove_item non siano indivisibili), i frammenti rilevanti dei thread A e B sono i seguenti:

```
A (produttore):
........
While (true) {
      item = produce_item();
      insert(item);
      }
........

B (consumatore):
........
While (true) {
      item= remove_item();
      consume(item);
      }
........
```

Esercizio 46)

risolvere il problema dei lettori/scrittori in moda che possa causare l'attesa indefinita (*starvation*) ma non lo stallo (*deadlock*) dei processi componenti.

Nel problema dei processi lettori/scrittori può avvenire il fenomeno di attesa indefinita; infatti, se mentre è presente un processo lettore nella sezione critica, continuano ad arrivare altri lettori, tutti i processi scrittori dovranno aspettare (attesa indefinita) l'uscita dell'ultimo lettore dalla sua sezione critica, rilasciando così la mutua esclusione sul semaforo db.

Esercizio 47)

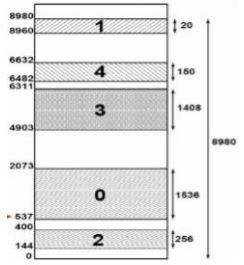
Si realizzi mediante semafori,
In una mensa universitaria, gli studenti, dopo aver mangiato depongono i vassoi in M contenitori (solo uno studente alla volta può usare un contenitore), ognuno di K ripiani. Periodicamente, un addetto alle cucine sceglie un contenitore tra quelli in cui non ci sono più ripiani liberi, lo svuota, lava i piatti e riporta il contenitore in sala. Implementate due processi: cameriere e studente. La soluzione deve essere tale che se uno studente sceglie il contenitore che l'addetto sta liberando (e lavando) aspetta (con il vassoio in mano) che l'addetto lo riporti in sala.

Esercizio 48)

Data una memoria di 4 page frame, inizialmente liberi, e 8 pagine di memoria virtuale. Utilizzando FIFO per il rimpiazzo delle pagine, indicare quanti page fault avvengono a fronte della stringa di riferimento: 0 1 4 7 3 7 4 1 0 3
Mostrare procedimento mostrato per arrivare alla soluzione.

Esercizio 49)

Data la seguente allocazione in memoria, determinare indirizzo fisico dei vari indirizzi virtuali



indirizzo virtuale
$< 0, 1984 >$
$< 1, 18 >$
$< 2, 250 >$
$< 3, 1400 >$
$< 4, 112 >$

Esercizio 50)

Si consideri la matrice (o *array* bidimensionale):

```
int A[][] = new int[100][100];
```

Si assuma che la posizione `A[0][0]` di tale matrice sia posta alla locazione **200** di una memoria paginata con **3** pagine di dimensione **200** B.
Si assuma che un valore `int` occupi **1** B.
Si assuma che le pagine siano inizialmente tutte vuote e che il processo (il cui codice occupa esattamente **200** B) abbia la seguente esecuzione:

```
for (int i = 0; i < 100; i++){
   for (int j = 0; j < 100; j++){
      A[i][j] = 0;}}
```

Quanti *page fault* saranno generati usando LRU?

N.B. ogni pagina contiene 200 elementi della matrice

La matrice `A[i][j]` è scritta linearmente in memoria:

`A[0][0],A[0][1],A[0][2],…,A[0][99],A[1][0], A[1][1],…,A[99][99]`

Il processo azzera le celle della matrice proprio nel loro ordine di memorizzazione.

Quindi inizialmente verranno caricati nelle pagine
Pagina 0: Il programma (che occupa esattamente 200 B)
Pagina 1: Celle da A[0][0] a A[1][99] incluse
Pagina 2: Celle da A[2][0] a A[3][99] incluse

ogni page fault carica elementi contigui per riempire la pagina

Esercizio 51)

Si consideri un sistema dotato di memoria virtuale, con memoria fisica divisa in 8 *page frame* condivisa da 4 processi contemporaneamente attivi: A, B, C e D. Si supponga che all'istante 100 lo stato della memoria sia il seguente (con il campo riferita avente SI = 1 e NO = 0):

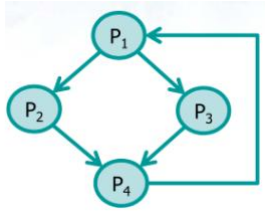
processo	pag. logica	pag. fisica	istante caricam.	R
A	0	7	50	1
A	1	6	37	0
B	5	5	30	1
B	3	4	97	0
C	2	0	15	1
C	5	2	70	0
C	9	1	92	0
D	8	3	27	0

Globale implica che considera tutte le pagine di un dato processo, e.g. A ha due pagine di cui una riferita (R), dunque se controllate A lo considerate riferito (ho una pagina con R=1), poi ponete tutte le sue pagine a R=0 e considerate il processo successivo.

Si supponga che il sistema utilizzi algoritmo di sostituzione second chance globale, mostrare come si modifica lo stato della memoria dopo le seguenti operazioni:
istante 101, C riferisce la pagina logica 5
istante 105, A riferisce la pagina logica 9

Esercizio 52)

Si realizzi mediante semafori il seguente grafo di precedenza, utilizzate 4 semafori, nell'istante iniziale solo P1 può essere eseguito.
Prima inizializzate i semafori, poi le funzioni per gestire i processi ciclici.



Esercizio 53)

Si consideri un file system FAT 32 su un disco da 1 GB con cluster da 2 KB. Il disco è pieno complessivamente al 10%, e la metà dello spazio è occupato da file di dimensione inferiore a 2 KB. Stimare il numero dei file di dimensione inferiore a 2 KB.

Esercizio 54)

Sia dato un sistema operativo in esecuzione su un computer con memoria centrale organizzata in celle di 16 bit. Si assuma che i processi utente vengano caricati all'interno di uno spazio di 4 Kbyte di memoria centrale, allocata in modo contiguo, selezionando un'area libera tramite l'algoritmo *best fit* e allocando i processi nella sua parte iniziale, riducendo l'area libera alla memoria che avanza. Analogamente, quando un processo termina la memoria che questo occupava va a formare un'area libera, fondendo assieme eventuali aree libere contigue.

Processo	Grandezza	Tempo di esecuzione	Durata
P_1	2 Kbyte	0	10
P_2	10^{-3} Mbyte	7	13
P_3	500 byte	11	19
P_4	2400 bit	13	10

1. Quali aree di memoria risultano occupate e quali libere subito dopo che ogni processo è stato caricato in memoria? E subito dopo che ognuno dei processi termina?

[1]Le conversioni sono tutte precise, a parte quella relativa al processo P_2 per cui si ha 10^{-3} Mbyte = $10^{-3} \cdot 1024 \cdot 1024$ byte = 1048, 576 byte, arrotondati a 1049 byte.

Processo	Grandezza (byte)	Tempo di esecuzione (ms)	Durata (ms)
P_1	2048	0	10
P_2	1049	7	13
P_3	500	11	19
P_4	300	13	10

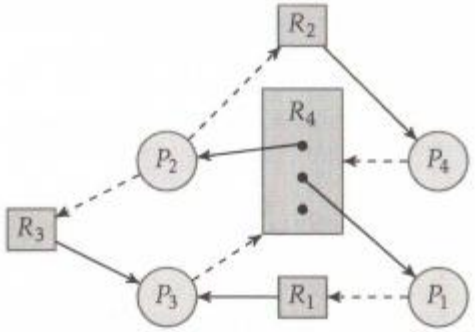
Esercizio 55)

Partendo dalla soluzione dell'esercizio precedente, rispondere alle seguenti domande:

Qual è la dimensione massima per un ipotetico processo in esecuzione all'istante $t = 10$ms?

Come cambierebbe la risposta al punto precedente se l'ipotetico processo venisse eseguito all'istante $t = 13$ms?

Esercizio 56)



Il sistema si trova in uno stato di *deadlock*? Perché?

Utilizzando l'algoritmo del banchiere, decidere se sia possibile o meno soddisfare una richiesta del processo P_3 di accedere alla risorsa R_4.

Esercizio 57)

Si consideri un sistema operativo dotato di memoria virtuale con paginazione a richiesta pura e rimpiazzamento con algoritmo LRU (*least recently used*), abbinato a uno schema di prepaginazione che carica le prime pagine di un processo fino a riempire i *frame* disponibili. Si presupponga di eseguire un processo che richieda, nell'ordine, di accedere alle seguenti pagine:

$$15, 15, 0, 2, 3, 11, 2, 6, 4, 14.$$

Avendo a disposizione 4 *frame* $f_0, \ldots, f_3$ cui associare le pagine del processo, e ipotizzando che quando vi siano più *frame* in cui inserire una pagina questa venga assegnata al *frame* f_i con il minimo indice possibile, si calcoli il numero di *page fault* che si verificano durante l'esecuzione del processo.

Svolgimento: Fissando convenzionalmente a $t = 0$ il tempo in cui il processo viene caricato in memoria, in tale istante lo schema di prepaginazione inserisce le pagine da 0 a 3 nei *frame* del processo.

Esercizio 58)

Si consideri un sistema operativo dotato di memoria virtuale con paginazione a richiesta pura e rimpiazzamento con algoritmo ottimale. Presupponendo di eseguire un processo che richieda, nell'ordine, di accedere alle seguenti pagine

$$6, 6, 2, 12, 7, 2, 0, 12, 2, 6,$$

rispondere alle domande che seguono.

1. Avendo a disposizione tre *frame* $f_0, \ldots, f_2$ cui associare le pagine del processo, e ipotizzando che quando vi siano più *frame* liberi in cui inserire una pagina questa venga assegnata al *frame* f_i con minimo indice possibile, si calcoli il numero di *page fault* che si verificano durante l'esecuzione del processo.

2. Se il numero di *frame* associati al processo aumentasse, il numero di *page fault* potrebbe diminuire? Perché?

3. La risposta alla domanda precedente potrebbe cambiare se le richieste di accesso alle pagine fossero state diverse? Perché?

Esercizio 59)

Si consideri un sistema operativo in cui la memoria centrale è allocata in modo distribuito ai processi tramite la tecnica della paginazione, utilizzando nel contampo tecniche di memoria virtuale. Si assuma che la memoria fisica sia divisa in *frame* di un Kbyte, che ogni indirizzo di memoria fisica occupi 32 bit e che in un indirizzo logico i bit più significativi indichino il numero di pagina e quelli meno significativi indichino lo scostamento.

1. Quanti bit sono necessari per specificare lo scostamento all'interno di una pagina?

2. Ipotizzando che anche l'indirizzo di una cella di memoria occupi 32 bit, qual è la quantità massima di memoria indirizzabile?

3. Si consideri la tabella delle pagine di un ipotetico processo

N. di pagina	bit di validità	N. di *frame*
0	1	739
1	1	127
2	0	–
3	1	91
4	0	–
5	1	470
6	1	2
7	1	106
8	1	495
9	0	–

Specificare che cosa succede quando il processo tenta di accedere ai seguenti indirizzi logici (espressi in notazione binaria, dividendo i bit in quartetti e abbreviando gli zero iniziali, al fine di facilitare la leggibilità), indicando ove possibile quali siano gli indirizzi fisici corrispondenti, esprimendoli in notazione decimale:

(a) $(0\ldots0\ 0111\ 1101\ 0101)_2$,

(b) $(0\ldots0\ 0100\ 0001\ 0111)_2$,

(c) $(0\ldots0\ 1011\ 1101\ 1011)_2$.

Esercizio 60)

- Siano dati i semafori e i processi indicati
  - Quale ordine di esecuzione è possibile?

```
init (S1, 1);
init (S2, 0);
```

P_1

```
...
while (1) {
  wait (S1);
  SC P1
  signal (S2);
}
...
```

P_2

```
...
while (1) {
  wait (S2);
  SC P2
  signal (S2);
}
...
```

P_3

```
...
while (1) {
  wait (S2);
  SC P3
  signal (S1);
}
...
```

Esercizio 61)

For a given operating systems the processes are executed with preemptive scheduling and using a cyclic (round-robin) policy.

The following table shows, for each process, its related arrival time and execution time. All the processes only perform compute (cpu-intensive) operations.

Process	Arrival time	Execution time
P1	0	500
P2	100	300
P3	300	400
P4	600	1000
P5	700	600

We want to evaluate the effect in the scheduling when the time slice length is changed. The considered time slice values are 200 and 500 miliseconds.

For each considered value, complete the following tasks:

1. Determine at what time ends each process.
2. Determine the time that each process was running (turnaround time).
3. Determine the service and wait time for each process of each process.
4. Determine the normalized turnaround time.
5. Determine the average waiting time.
6. Determine the average turnaround time.

Esercizio 62)

A virtual memory system with on-demand paging uses a page size of 512 words, a virtual memory of 16 pages numbered from 0 to 15, and a physical memory of 4 frames numbered from 0 to 3. The current content of memory is:

#Frame	Pege
0	Pág. 4 of process
1	Pág. 9 of process
2	Pág. 5 of process
3	Pág. 1 of process

a) Show the content of the page table.
b) Physical addresses equivalent to logicals 1628, 851, 2700 y 2432.
c) What does it happen when logical address is used 1330?

Esercizio 63)

Given a hard disk with a Unix-like filesystem with the following specifications: the block size is 4KB, the block address length is 4 bytes and the i-nodes have a traditional structure (10 direct pointers, 1 single indirect pointer, 1 double indirect pointers and 1 triple indirect pointer).

Answer the following questions:

a) What is the number of blocks (including both the data and address blocks) that have the following files:
   - File A with a size of 20 KBytes
   - File B with a size of 200 KBytes
   - File C with a size of 2000 KBytes
   - File D with a size of 20000 KBytes
b) What is the maximum file size?

Esercizio 64)

Tre processi P1 P2 e P3 arrivano uno dopo l'altro – nell'ordine indicato ma in tempi diversi – in coda di ready. I tre processi sono caratterizzati da un unico burst di CPU e da nessun burst di I/O. In quale caso il waiting time medio del sistema relativo ai tre processi risulta lo stesso sia adottando un algoritmo di scheduling FCFS che adottando una algoritmo di scheduling SJF preemtpive?

Esercizio 65)

Un hard disk ha la capienza di 2^{40} byte, ed è formattato in blocchi da 1024 byte. Quanti accessi al disco sono necessari per leggere l'ultimo blocco di un file A della dimensione di 2^{15} byte, assumendo che sia già in RAM il numero del primo blocco del file stesso e che venga adottata una allocazione concatenata dello spazio su disco? (motivate la vostra risposta) PS ogni puntatore al blocco successivo occupa 4 byte.

Esercizio 66) ricordate che possiamo allocare un byte alla volta, e.g. se indirizzo occupasse 9 bit allora necessiterà di due byte

In un sistema operativo un indirizzo fisico è scritto su 29 bit, e un frame è grande 8 Kbyte. E' noto che la tabella delle pagine più grande del sistema occupa esattamente un frame.

a) Qual è lo spazio di indirizzamento logico del sistema (esplicitate i calcoli che fate)?

Esercizio 67)

All'atto dell'installazione di un SO su una macchina, è possibile scegliere se usare pagine grandi 2^{10} byte oppure grandi 2^{20} byte. La dimensione dello spazio di indirizzamento logico rimane comunque la stessa. La macchina su cui viene installato il SO usa 32 bit per scrivere un indirizzo fisico, e lo spazio di indirizzamento fisico è la metà dello spazio di indirizzamento logico (nel seguito, motivate tutte le risposte che date, esplicitando tutti i calcoli ed eventuali assunzioni fatte).

a) Nel caso di pagine da 2^{20} byte, quanto può essere grossa, al massimo, la page table di un processo?

Esercizio 68)

in un negozio di barbiere ci sono
- una sedia per lavorare
- n sedie per far attendere i clienti

- il barbiere di solito dorme
- quando arriva un cliente, questi lo sveglia e si fa servire
- se nel frattempo arrivano altri clienti, si accomodano sulle sedie oppure vanno via se sono tutte occupate

Implementare mediante semafori usando le seguenti variabili

```
#define CHAIRS 5                /* # chairs for waiting customers */

typedef int semaphore;          /* use your imagination */

semaphore customers = 0;        /* # of customers waiting for service */
semaphore barbers = 0;          /* # of barbers waiting for customers */
semaphore mutex = 1;            /* for mutual exclusion */
int waiting = 0;                /* customers are waiting (not being cut) */
```

Esercizio 69)

All'atto dell'installazione di un SO su una macchina, è possibile scegliere se usare pagine grandi 2^{10} byte oppure grandi 2^{20} byte. La dimensione dello spazio di indirizzamento logico rimane comunque la stessa. La macchina su cui viene installato il SO usa 32 bit per scrivere un indirizzo fisico, e lo spazio di indirizzamento fisico è la metà dello spazio di indirizzamento logico (nel seguito, motivate tutte le risposte che date, esplicitando tutti i calcoli ed eventuali assunzioni fatte).

Assumendo pagine di 2^{10} byte, il sistema potrebbe dover usare una paginazione a più livelli? I.e. controllate dimensione Page Table e frame.

Esercizio 70)

- Considerate la seguente successione di riferimenti di pagine:
  1, 2, 3, 4, 2, 1, 5, 6, 2, 1, 3, 2, 7, 6, 3, 2, 1, 2, 3, 6.
- Determinare il tempo di accesso effettivo della paginazione su richiesta per LRU con 5 blocchi, se:
  - il tempo medio di servizio di un page fault senza salvataggio della pagina avvicendata é di 80 millisecondi ($80*10^{-3}$ sec),
  - il tempo medio di servizio di un page fault con salvataggio della pagina avvicendata è di 140 millisecondi ($140*10^{-3}$ sec)
  - il tempo di accesso alla memoria è di 80 microsecondi ($80*10^{-6}$ sec),
- nell'ipotesi che l'accesso alle pagine 1, 2 , 3 sia sempre in scrittura.

Nelle soluzioni specificate chiaramente il numero di page fault con salvataggio, senza salvataggio e il numero di accessi diretti senza page fault

Inizialmente i 5 blocchi sono vuoti, il tempo effettivo è la media, dunque totale diviso 20 (numero riferimenti)

i.e. se riferimento a tale pagina vuol dire che essa sarà scritta, dunque dovete salvarla quando "scaricata" dalla memoria

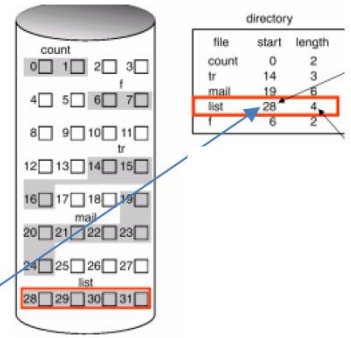
Esercizio 71)

Si consideri un file formato da 70 record e le sue possibili allocazioni su disco di tipo contigua, concatenata e con tabella indice. In ognuno di questi casi i record del file sono memorizzati uno per blocco.

Le informazioni che riguardano il file sono già in memoria centrale e la tabella indice è contenuta in un unico blocco.

Si dica quanti accessi al disco (letture e/o scritture) sono necessari, in ognuna di queste situazioni, per cancellare (eliminare):
1. il primo record
2. il 40-esimo record
3. l'ultimo record

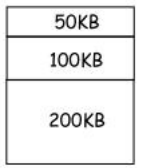
- per cancellare il primo record e' sufficiente modificare il puntatore al blocco di inizio ➔ 0 accessi al disco



Esercizio 72)

Un programma ha il segmento di testo di 200KB, quello dei dati di 100KB e uno stack di 50KB.

Se la memoria è organizzata a pagine da 16KB:
1. Quanto è lunga la tabella delle pagine del processo?
2. Quanta memoria può al più essere gestita se si hanno a disposizione 32 bit per l'indirizzamento?



Ogni segmento del processo e' paginato

Esercizio 73)

- Supponiamo che il sistema di paginazione usato dal S.O. assegni 3 frame da 512B a ciascun processo e si consideri il seguente programma

```
...
#define N  512
int A[N], C[N];
int i,j;
...
for(i=1; i<=2; i++)
for(j=0; j<N/2; j++)
A[i*j] = A[2*i] + C[2*j];
...
```

per calcolare riferimenti considerate il seguente ordine:
a[2i] C[2j] a[ij]

1. si determini la stringa dei riferimenti alle pagine della memoria supponendo che un intero si rappresentato con 2B
2. si determini la tabella delle pagine, al termine dell'esecuzione della procedura, nel caso che l'algoritmo di avvicendamento sia (i) FIFO, (ii) LRU

- Gli array A e C occupano 1024B ciascuno, suddivisi in 2 pagine da 512B come segue:
  - A[0..255] alla pagina 0, A[256..511] alla pagina 1
  - C[0..255] alla pagina 2, C[256..511] alla pagina 3

Esercizio 74)

Illustrare la composizione (memoria complessiva) di un sistema RAID con 8 GB di memoria secondaria effettivamente utilizzabile con riferimento a :

a) Configurazione di livello 0 e 4 dischi
b) Configurazione di livello 1 e 4 dischi
c) Configurazione di livello 4 e 5 dischi
d) Configurazione di livello 5 e 3 dischi

avete a disposizione solo
hard disk di 2 GB
o 4 GB

Esercizio 75)

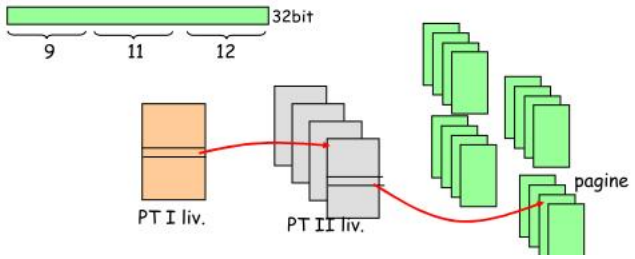
Utilizzare due semafori (*bus* inizializzato a 30 e *door* inizializzato a 2) per gestire il seguente problema:
ogni bus ha capacità di 30 passeggeri e ha due porte, gestire il fatto che non ci possono essere più di 30 passeggeri e che i passeggeri non devono scontrarsi quanto entrano o escono dal bus.

Esercizio 76)

Un computer ha indirizzi a 32 bit, e pagine su due livelli. Gli indirizzi virtuali sono composti da:

- 9 bit per l'indice della page table (PT) principale
- 11 bit per lo spostamento all'interno della PT di secondo livello
- 12 bit per l'offset nella pagina

Quanto sono grandi le pagine e quante pagine virtuali ci sono?



Esercizio 77)

- Supponiamo che il sistema di paginazione utilizzato dal sistema operativo assegni 3 frame (blocchi di memoria) da 512B a ciascun processo e che l'algoritmo di sostituzione delle pagine sia LRU.
- Prendiamo in considerazione il seguente programma:

```
...
#define N 512
int a[N];
int i;
...
for (i=0; i < N/2; i++)
a[i] = a[2*i] + a[N-i-1];
...
```

per calcolare riferimenti considerate il seguente ordine:
a[2i] a[N-i-1] a[i]

- Si risponda ai seguenti quesiti:
  - se la dimensione di un intero è 4B, qual è il numero di page faults ?
  - In tal caso, se il tempo medio di servizio di un page fault é di 25 millisecondi ($25*10^{-3}$ sec) ed il tempo di accesso alla memoria di 100 microsecondi ($100*10^{-6}$ sec), qual è il tempo di accesso medio della paginazione su richiesta?

Le pagine sono da 512B e ogni intero è 4B (128 elementi a pagina):

0	1	2	3
a[0] ... a[127]	a[128] ... a[255]	a[256] ... a[383]	a[384] ... a[511]

Esercizio 78)

Considerare un sistema con memoria virtuale e segmentazione con parola da un byte. Supponendo di avere indirizzi logici di 10 bit, i cui primi 4 più significativi indicano il numero di segmento, dire:

- (a) qual è la massima grandezza possibile per un segmento;
- (b) di quanti segmenti al più può essere composto un processo.

Supponendo che il processo attualmente in esecuzione sia composto da 5 segmenti S_0, S_1, S_2, S_3 ed S_4 e che, a un dato istante, la sua tabella dei segemnti sia la seguente:

Numero di segmento	Lunghezza	Base
0	10	200
1	20	100
2	6	252
3	32	720
4	32	683

Ogni segmento ha dimensione pari a quella trovata nel punto a)

9, dunque segmento 0 e offset 9, segmento 0 ha base 200, dunque 200+9

tradurre in indirizzi fisici i seguenti indirizzi logici: 9, 132, 79, 64, 259, 135, 320.

Esercizio 79)

Un incrocio di due grandi viali (uno in direzione Nord-Sud, l'altro in direzione Est-Ovest) è regolato da un vigile. Il vigile lascia passare le macchine su uno dei due viali (in ambedue i sensi di percorrenza) per un tempo T, poi blocca il transito da quel viale, attende che l'incrocio si liberi dagli automezzi che lo avevano già impegnato, e quindi dà via libera ai mezzi di trasporto in attesa sull'altro viale, lasciandoli transitare per un tempo T, e così via indefinitamente. I due viali sono così ampi che nell'incrocio non si verifica mai un ingorgo ed il flusso di traffico è sempre scorrevole. Fornito il pseudo-codice dei generici processi "automezzo sulla direttrice Nord-Sud" (auto_NS) e "automezzo sulla direttrice Est-Ovest" (auto_EW), considerando che all'avvio del vigile l'impegno dell'incrocio dovrà essere già stato assegnato alle macchine in transito sulla direttrice Nord-Sud, implementate la procedura che implementa il vigile.

Attenzione, ricordate che "vigile lascia passare… per un tempo T"

```
shared int n = 0;
shared semaphore NS = 1;
shared semaphore EW = 0;
shared semaphore mutexNS = 1;
shared semaphore mutexEW = 1;
shared semaphore incrocio = 1;
shared semaphore conta = 1;

auto_NS() {
     int mn;

     wait(mutexNS);
     wait(NS);
     signal(mutexNS);

     wait(conta);
     mn = ++n;
     signal(conta);
     if (mn == 1)
        wait(incrocio);
     signal(NS);

     passa_incrocio();

     wait(conta);
     mn = --n;
     signal(conta);
     if (mn == 0)
        signal(incrocio);
}

auto_EW() {
     int mn;

     wait(mutexEW);
     wait(EW);
     signal(mutexEW);

     wait(conta);
     mn = ++n;
     signal(conta);


     if (mn == 1)
        wait(incrocio);
     signal(EW);

     passa_incrocio();

     wait(conta);
     mn = --n;
     signal(conta);
     if (mn == 0)
        signal(incrocio);
}
```

Note:
a) L'inizializzazione assicura che all'inizio abbiano il via le macchine sulla direttrice NS

b) mutexNS e mutexEW fanno si che una sola macchina per volta si accodi sui semafori NS ed EW, in modo che il vigile possa bloccare al più presto il semaforo allo scadere del tempo, senza dover aspettare molte macchine già in coda (importante curare questo aspetto).

c) Il semaforo incrocio consente al vigile di aspettare che l'incrocio sia libero prima di dare il via libera sull'altra direzione (questo requisito è importante).

Esercizio 80)

Data la seguente stringa di riferimenti

1, 2, 3, 4, 2, 1, 5, 6, 2, 1, 2, 3, 7, 6, 3, 2, 1, 2, 3, 6.

Quanti page fault avvengono usando LRU e algoritmo ottimo (supponendo sistema dotato di cinque frames)?

Esercizio 81)

Considerando spazio degli indirizzi logici di 64 pagine da 1024 parole (byte) cadauna, mappato in una memoria fisica di 32 frames (1024 parole per ogni frame).
Quanti sono i bit degli indirizzi logici? Quanti degli indirizzi fisici?

Esercizio 82)

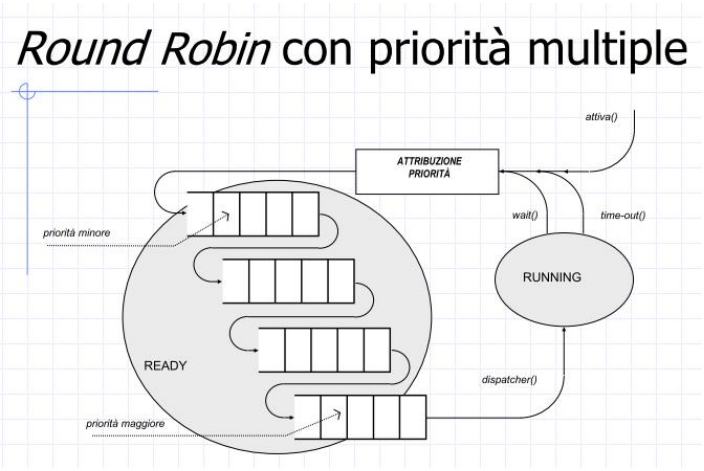
Un dato sistema operativo possiede indirizzi virtuali di 21 bit, viene installato in un device con indirizzi fisici a 16 bit, le pagine hanno dimensione 2 KB.
Quanti saranno gli elementi in una tabella delle pagine? Quanti in tabella delle pagine invertita?
Quant'è la massima memoria utilizzabile dal dato sistema operativo nel dato device?

Esercizio 83)

Dato il seguente scheduling, usando round robin preemptive di 10 unità, esiste inoltre un idle task con 0 Burst (idle task quando non ci sono processi pronti). Se un processo è preempted allora è posizionato in fondo alla coda wait.
Mostrare il diagramma Gantt dello scheduling.
Qual è turnaround di ogni processo?
Qual è waiting time di ogni processo?

Process	Priority	Burst	Arrival
P_1	40	20	0
P_2	30	25	25
P_3	30	25	30
P_4	35	15	60
P_5	5	10	100
P_6	10	10	105

Round Robin con priorità multiple



Esercizio 84)

Supponiamo di avere un processo di n frames e una stringa di riferimenti di lunghezza p (tutte le n pagine vengono riferite). Indipendentemente dall'algoritmo utilizzato, qual è lower bound e upper bound del numero di page fault? Motivare la risposta.

Esercizio 85)

Si vuole modellare un sistema composto da una macchinetta distributrice di aranciata, un fornitore di aranciata e N = 20 consumatori di aranciata.
La macchinetta fornisce due operazioni:

a) caricamento, che permette al fornitore di rifornire la macchinetta con M=10 aranciate;
b) prelievo, che permette ad un consumatore di prelevare una aranciata.

Il fornitore inizialmente carica la macchinetta, poi si sospende in attesa di essere chiamato a rifornire nuovamente la macchinetta vuota. Ogni consumatore ha accesso alla macchinetta per prelevare aranciate.
Il primo consumatore che trova la macchinetta scarica deve chiamare il fornitore e attendere una nuova disponibilità di aranciata. Finché la macchinetta è vuota i consumatori si devono sospendere in attesa di aranciata.
Scrivere le procedure carica e preleva che simulano la situazione utilizzando i seguenti semafori e variabili:

```
type A = array[1..10] of orange;
var pieno: semaphore(initial 1);
    vuoto: semaphore(initial 0);
    mutex: semaphore(initial 1);
    index: 1..10;
    aranciate: A;
```

Index conta il numero delle arance
Pieno serve per svegliare il fornitore
Vuoto per dire a chi effettua il prelievo che si sono arance disponibili

Esercizio 86)

Si consideri uno spazio di indirizzamento logico di otto pagine di 1024 parole ognuna, mappate su una memoria fisica di 32 frame.

(a) Da quanti bit è costituito l'indirizzo logico?

(b) Da quanti bit è costituito l'indirizzo fisico?

Esercizio 87)

Si consideri la seguente stringa di riferimenti (partendo con $t = 0$):

2 6 5 7 7 7 7 5 1 6 4

working set $WS(t, \Delta)$ all'istante t con intervallo Δ.

Cosa è $WS(10, 8)$, ossia dopo l'ultimo accesso?

Nel precedente esempio, quanti page fault ci sono complessivamente con $\Delta = 4$ (supponendo che in ogni istante si mantenga in memoria esattamente il solo working set)?

Esercizio 88)

Si consideri un disco a 7.200 RPM (rotazioni per minuto), con tempo medio di seek pari a 3 ms ed un transfer rate di 40 MB/s. Calcolare il tempo medio necessario a leggere 4KB da una traccia t. Si calcoli poi il tempo medio necessario a soddisfare una richiesta di 4KB sempre relativa alla stessa traccia t (supponendo che la testina non si sia ancora mossa dalla traccia t in questione). Nei calcoli si trascurino i tempi dovuti all'esecuzione delle routine del kernel.

$\frac{60 \cdot 10^3}{2 \cdot 7200} \equiv 4,17$

Esercizio 89)

. Si consideri un file system Unix-like (UFS o EXT2) con blocchi da 4K, su un disco con $t_{seek} = 10ms$, a 7200 RPM. In tale file system, sia stato aperto un file i cui blocchi siano sulla stessa traccia del rispettivo inode.

(a) Quanto si impiega per accedere direttamente alla posizione 10000 del file?

(b) e alla posizione 100000?

Dunque, in ogni blocco ci sono 4000 posizioni del file.
Attenzione, non sapete posizione attuale della testina del disco, non avete tale informazione.

Esercizio 90)

Si consideri la seguente *segment table*:

Segmento	Base	Lunghezza
0	219	600
1	2300	14
2	90	100
3	1327	580
4	1952	96

Quali sono gli indirizzi fisici corrispondenti ai seguenti indirizzi logici?

- 0, 430
- 1, 10
- 2, 500
- 3, 400
- 4, 112

Esercizio 91)

Dato il seguente codice:

```
print() {

  process P1 {
    write(''line 1''); write(''line 2'');
  }

  process P2 {
    write(''line 3''); write(''line 4'');
  }

  process P3 {
    write(''line 5''); write(''line 6'');
  }

}
```

aggiungete semafori (specificando la loro inizializzazione) per ottenere il seguente output: 1 2 3 4 5 6

Per semplicità potete presupporre che esista singola istanza di P1, P2 e P3

Esercizio 92)

Può avvenire deadlock? Come? Quali sono i valori possibili di x,y e z in caso di deadlock? Motivate le risposte

```
int x = 0, y = 0, z = 0;
sem lock1 = 1, lock2 = 1;

process foo {
  z := z + 2;
  P(lock1);
  x := x + 2;
  P(lock2);
  V(lock1);
  y := y + 2;
  V(lock2);
}

process bar {
  P(lock2);
  y := y + 1;
  P(lock1);
  x := x + 1;
  V(lock1);
  V(lock2);
  z := z + 1;
}
```

Esercizio 93)

Dato un sistema con indirizzi logici a 32 bit e pagine da 4KB, sappiamo inoltre che il sistema riesce a supportare 512 MB di memoria fisica
Quante entries ci saranno in una convenzionale tabella delle pagine a singolo livello? Quante in una tabella delle pagine invertita?

Esercizio 94)

Un disco possiede un tempo medio di servizio pari a 15 ms. nel caso in cui si escluda la cache;
se invece la cache è attiva, con una hit ratio (probabilità di trovare il dato in essa) pari a 80%, il tempo di servizio è di 6 ms.

Calcolare il valore che deve avere tale probabilità perché il tempo di servizio diventi 4.2 ms. a parità delle altre condizioni.

Tempo di servizio è dato dalla somma del tempo di servizio del disco (considerando i casi nei quali non si accede alla cache) e del tempo di servizio della cache (quando il dato si trova nella cache).

Per semplificare i calcoli, potete non inserire la componente “tempo di accesso alla cache quando c’è miss e dunque accedo alla memoria secondaria”, basta che inseriate il tempo della memoria secondaria se avviene miss; considerate tempo cache solo quando non c’è il miss (i.e. dato presente nella cache).

Esercizio 95)

Considerare una politica di Disk Scheduling gestita tramite l’algoritmo SCAN. Si supponga, per semplicità, un tempo di seek lineare col numero di tracce attraversate dalla testina e un ritardo rotazionale e un tempo di trasferimento dati approssimabili con 0. Il tempo di seek è di una traccia al millisecondo, la testina al tempo 0 si trova sulla traccia numero 50 e il disco è composto da 200 tracce (da 0 a 199). Una volta che viene avviata una richiesta di I/O al disco non è possibile interromperla, per cui le scelte operate dallo scheduler avvengono soltanto dopo il completamento di una richiesta di I/O. Si supponga, infine, che, se non ci sono richieste pendenti, la testina rimanga ferma sull’ultima traccia visitata e che la direzione di scansione venga invertita quando non ci sono ulteriori richieste pendenti nella direzione attuale.

Con riferimento alla tabella 1, che indica i tempi di arrivo in millisecondi di varie richieste di I/O su disco e le relative tracce, fornire:

- (a) la risultante sequenza con cui esse vengono soddisfatte dallo scheduler e i relativi tempi;

Tabella 1

Numero richiesta	Traccia	Tempo d’arrivo
1	70	2
2	65	8
3	80	20
4	75	21
5	120	29
6	60	31
7	90	38
8	80	100

Esercizio 96)

Considerare il seguente pseudo-codice:

```
const int n = 20;
int conta;

void incrementa() {
    int i;

    for(i = 0; i<n; ++i {
        int t;

        t = conta;
        conta = t + 1;
    }
}

void main() {
    conta = 0;
    parbegin(incrementa, incrementa);
    write(conta);
}
```

che avvia due processi concorrenti indipendenti i quali incrementano iterativamente una variabile condivisa. Sapendo che la velocità relativa di avanzamento dei due processi non è in alcun modo prevedibile:

1. quali sono i valori minimo e massimo che il risultato può assumere e perché?
2. quali sono i valori minimo e massimo che il risultato può assumere se vengono avviati M processi invece di 2 e perché?

Esercizio 97)

Data un file system con inode di 1kB, 16 indici diretti di blocco, 1 blocco con 1 livello di indirezione e 1 blocco con 2 livelli di indirezione. Qual è la dimensione massima di un file in tale sistema?
Ogni puntatore a un blocco occupa 4B, anche i blocchi della memoria occupano 1kB

Esercizio 98)

Sia dato un disco di 16 GB diviso in blocchi ampi 1 KB. Si considerino due possibili strutture per tener traccia dei blocchi liberi: lista concatenata e *bitmap*. Nel primo caso, ogni elemento della lista è costituito a sua volta da un blocco, il quale contiene indici di blocco (di 32 *bit* ciascuno), dei quali l'ultimo è riservato per l'indicazione del prossimo blocco di lista libera. Nel secondo caso l'uso di un *bit* 1 o 0 definisce se il corrispondente blocco sia libero o utilizzato.
Si calcoli l'occupazione di memoria delle due strutture.

Ogni blocco può contenere 1 KB / 4 B/indice = 256 indici di blocco di cui 1 viene usato come collegamento al "blocco di indici" successivo nella lista. Ne rimangono dunque 255 utilizzabili per rappresentare i blocchi liberi.

Esercizio 99)

- Sincronizzazione di 3 Processi coi Semafori.
- Si considerino i processi A, B e C che si sincronizzano come mostrato nel seguito attraverso i semafori Sem1, Sem2 e Sem3 (inizializzati a 0) e che operano sulle variabili condivise x, y e z, che sono inizializzate come segue: x = 1; y = 2; z = 1.
- Con quale ordine i processi stampano i valori delle tre variabili?
- Qual è il valore delle tre variabili che viene infine stampato?

Problema

Inizializzaz.

```
x = 1;
y = 2;
z = 1;
Sem1 = 0;
Sem2 = 0;
Sem3 = 0;
```

Process A {	Process B {	Process C {
P(Sem1);	P(sem2);	V(Sem2);
z = (x - z)*y ;	x = x + y;	P(Sem3);
x = x+z+y;	z = x - z;	x = x / y ;
V(Sem3);	y=(y-z)+x;	y = 2z + x ;
P(Sem1);	Print(y);	V(Sem1);
x = x + y;	V(sem1);	z = x + z;
Print(x);	}	Print(z);
}		}

Esercizio 100)

In un sistema le pagine hanno una dimensione di 1kB (1024), e la RAM è suddivisa in 256 frame. Nelle page table, un numero di pagina è scritto su 2 byte, e la page table più grande ammessa dal sistema occupa completamente un frame della RAM (per semplicità non consideriamo dirty bit e bit di validità).
qual è la lunghezza in bit di un indirizzo fisico?

Esercizio 101)

Si considerino tre processi (A, B e C) i quali devono eseguire alcune operazioni sulle variabili x e y e poi stamparne il risultato finale. A questo proposito, si consideri la seguente sequenza di avvenimenti:

- I processi A e C devono essere in attesa di essere risvegliati
- Il processo B sveglia il processo A, che a sua volta sveglia il processo C.
- I tre processi accedono concorrentemente (ma in mutua esclusione) alle variabili condivise x e y, poi B e C si bloccano in attesa che A termini la computazione su x e y (B e C avranno eseguito la loro computazione su x e y prima di bloccarsi)
- Terminata la computazione, è A stesso a svegliare B che a sua volta sveglia C.
- B stampa il valore di x e C stampa il valore di y (senza un ordine prestabilito).

[A] Assumendo che i semafori SemA, SemB e SemC siano inizializzati a 0, che il semaforo Mutex sia inizializzato a 1, e che inizialmente si abbia x = 2 e y = 1, si correggano e/o completino i frammenti di codice riportati nella slide seguente.

Process A {	Process B {	Process C {
	P(semB);	V(semA)
V(mutex);		
y= y + 2x ;	x= x + 1;	x= y * 3;
	V(mutex);	
P(SemA);	V(SemB);	V(SemC);
	Print(x);	Print(y);
}	}	}

Esercizio 102)

Dato un sistema di *swapping* e una memoria con zone disponibili di ampiezza: 8, 4, 14, 18, 17, 9, 12, 15 KB, in questo ordine, indicare quale area venga prescelta dalla politica ***Next Fit*** a fronte della richiesta di caricamento di un segmento di ampiezza 3 KB dopo aver caricato un segmento ampio 12 KB:

- Algoritmo Next-fit
  - Scandisce la memoria partendo dall'ultimo blocco allocato, e sceglie il primo blocco disponibile successivo sufficientemente grande.

Esercizio 103)

- Numerosi operai in una fabbrica preparano un unico prodotto con l'utilizzo di **nA** quantità del componente **A**, **nB** quantità del componente **B**. Un fattorino viene chiamato a riempire le quantità di **A** e **B** fino a **totA** e **totB** ogni volta che le loro quantità residue scendono sotto **nA** ed **nB**. Prima di iniziare a comporre il prodotto, ogni operaio si assicura di avere le quantità necessarie dei due componenti; viceversa chiama il fattorino e attende che arrivi con la scorta. Una volta terminato il suo compito il fattorino resta in attesa di essere richiamato. Una volta completato un prodotto ogni operaio inizia a prepararne un altro. Inizialmente le scorte di **A** e **B** sono piene e nessun prodotto è stato ancora preparato.

- Scrivere le seguenti procedure che propongono una soluzione a questo problema utilizzando i costrutti dei monitor.

Il monitor potrebbe utilizzare i seguenti dati:

operaio: variabile condition sulla quale gli operai si sospendono in attesa di essere riforniti di un determinato componente;
fattorino: variabile condition sulla quale il fattorino si sospende in attesa di essere mandato a comprare le scorte di un componente;
operai_in_attesa: intero che indica il numero degli operai in attesa di un componente;
nA, **nB**: costanti che indicano la quantità necessaria di ogni componente per la preparazione di un prodotto;
totA, **totB**: costanti che indicano il numero massimo di scorte per ogni componente;
qA, **qB**: interi che indicano la quantità di componenti **A** e **B** attualmente presenti in fabbrica.

```
monitor Fabbrica {
condition operaio, fattorino;
int qA, qB;
int operai_in_attesa;
```

Implementare:

Funzione invocata da ciascun operaio all'infinito

```
void prepara_prodotto {
```

Funzione invocata da fattorino

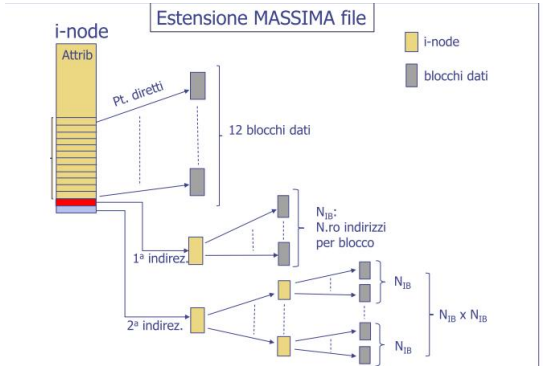
```
void scorta_ingrediente {
```

Esercizio 104)

- Si consideri una variante FAT-15 dell'architettura di file system nota come FAT-16, nella quale l'unica differenza dalla versione base sia che 1 bit dell'indice FAT non sia utilizzabile, tutti gli altri attributi di architettura rimanendo inalterati. Si specifichi la dimensione massima di file possibile con tale variante, e la dimensione massima di partizione.

Come in FAT-16 i blocchi su disco saranno ampi al massimo 32KB

Esercizio 105)

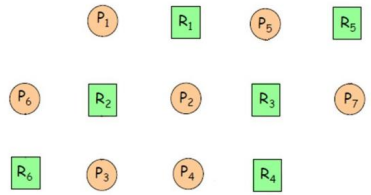


Usiamo i-node con dimensione blocco di 512 byte. Nodo indice primario contiene 12 campi di indirizzo di blocchi di disco e 2 puntatori a nodi indice di primo e secondo livello di indirezione. Indirizzi di blocco sono espressi su 32 bit; si vuole allocare un file logicamente composto da 10 000 record da 80 byte ciascuno, inoltre si impone che un record non possa essere suddiviso in due blocchi diversi. Quanti blocchi serviranno per allocare i dati? Quanti blocchi serviranno per gestire l'allocazione del file tramite nodi indice?

Esercizio 106)

Un sistema è composto da sette processi P1… P7 e da sei risorse condivise R1… R6 ciascuna diversa dalle altre, presente in singola istanza e ad accesso mutuamente esclusivo. La situazione corrente del sistema è la seguente:
- P1 occupa R1 e richiede R2;
- P2 non occupa risorse e richiede R3;
- P3 occupa R2;
- P4 occupa R4 e richiede sia R2 sia R3;
- P5 occupa R3 e richiede R5;
- P6 occupa R6 e richiede R2;
- P7 occupa R5 e richiede R4;

Si determini, utilizzando il grafo di allocazione delle risorse, se il sistema sia in stallo (*deadlock*) e, in caso affermativo, quali siano i processi e le risorse coinvolti.



Esercizio 107)

La seguente soluzione del problema dei lettori-scrittori contiene alcuni errori e mancanze. Lo studente ne modifichi il codice tramite aggiunte, cancellazioni e correzioni. Il risultato dovrà rappresentare una versione corretta, realizzata apportando il minor numero possibile di modifiche all'originale qui di seguito.

```
void Lettore (void) {
  while (true) {
    P(mutex);
    numeroLettori--;
    if (numeroLettori==1) P(db);
    V(mutex);
    // leggi il dato
    numeroLettori--;
    if (numeroLettori==0) P(db);
    // usa il dato letto
  }
}
```

```
void Scrittore (void) {
  while (true) {
    // prepara il dato da scrivere
    P(db);
    // scrivi il dato
    P(db);
  }
}
```

Esercizio 108)

In una chiavetta USB da 2 GB con filesystem FAT32 e blocchi da 4KB vengono registrati 100 files da 1000 blocchi ciascuno.

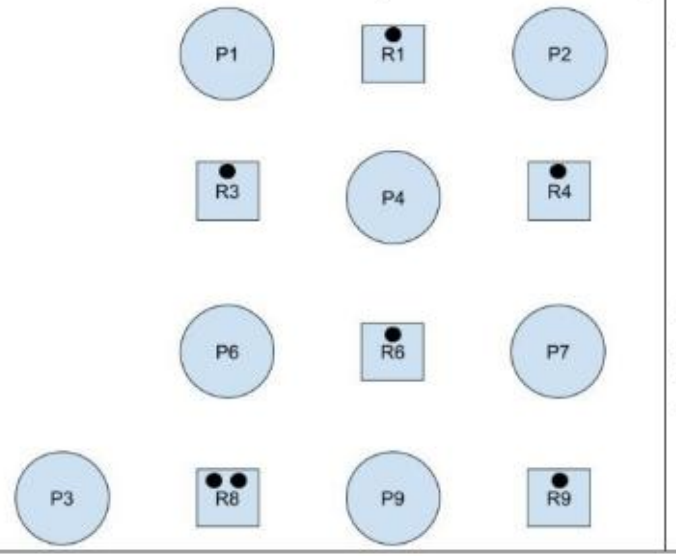
Considerando che:
il tempo di lettura di un blocco è 100 μs;
il tempo medio di accesso (tempo per raggiungere qualsiasi blocco da un altro qualsiasi, purché non contiguo altrimenti il tempo è nullo) è di 5 ms;
si presuma che la FAT sia precaricata in memoria (si trascurino pertanto i tempi di accesso alla stessa).

Calcolare il tempo necessario a leggere tutti i files, nelle ipotesi

a) di minima frammentazione;
b) di massima frammentazione;

Esercizio 109)

Un sistema è composto dai seguenti processi e risorse (di cui solo la risorsa R8 ha molteplicità 2 anziché 1)



La situazione corrente del sistema è la seguente:
P1 occupa R3, richiede R1
P2 occupa R1, richiede R4
P3 occupa una istanza di R8
P4 richiede R1
P6 occupa R6, richiede R3 e R8
P7 occupa R4, richiede R9
P9 occupa una istanza di R8 e R9, richiede R6

(Attenzione: questi sopra rappresentano una istantanea della situazione corrente e NON una sequenza cronologica. Dunque se leggete che P1 richiede R1, dovete lasciare la risorsa come NON assegnata).

Si determini, utilizzando il grafo di allocazione delle risorse, se il sistema sia in stallo (deadlock), motivate la risposta per TUTTI i cicli presenti.
Se presente deadlock specificate processi e risorse coinvolte.

Esercizio 110)

Supponiamo di avere due processi che condividono una variabile x e che i loro pseudo codici siano i seguenti.
PS: P = Wait, V = Signal.

P1	**P2**
1: P(SemA)	8: P(SemB)
2: P(Mutex)	9: P(Mutex)
3: if (x<0) x=x+1	10: x=x-2;
4: V(Mutex)	11: V(Mutex)
5: V(SemC)	12: V(SemA)
6: P(SemA)	13: P(SemC)
7: print(x)	14: P(Mutex)
	15: x=x-1
	16: V(Mutex)
	17: print(x)

I due processi P1 e P2 vengono eseguiti in maniera concorrente.
Si assuma:

- Valore iniziale di x sia 1;
- SemA e SemC inizializzati a 0;
- SemB e Mutex inizializzati a 1.

Si elenchi possibili ordini di esecuzione delle istruzioni (e.g. 1 8 2 10...), motivate la risposta.

Esercizio 111)

Sia data una partizione di disco ampia 64 GB, organizzata in blocchi dati di 1 KB e indici da 32 bit.
Si determini ampiezza massimo di file ottenibile usando i-node ampi esattamente un blocco, i-node principale contenente 12 indici di blocco, 1 blocco indice per singola indirezione (single indirect block) e un blocco indice per indirezione a 2 livelli (double indirect block). Quant'è lo spazio occupato dalla struttura i-node necessaria per rappresentare tale file?

Esercizio 112)

Considerate i due seguenti thread, eseguiti in concorrenza con memoria condivisa

Thread A	Thread B

`for (i=0; i<5; i++) {`
`  x = x + 1;`
`}`
	`for (j=0; j<5; j++) {`
	`  x = x + 2;`
	`}`

Supponiamo che x sia inizializzato a 0 prima che ogni thread inizi.
Inoltre supponiamo che l'operazione x=x+2 nel thread B sia atomica (e.g. come se fosse una funzione atomicIncr2(x)), questo non vale per x=x+1 del thread A.
Quali sono i possibili valori finali di x? Motivare risposta

Esercizio 113)

Un file è dotato di una struttura a record di 512 byte ed è allocato su un disco caratterizzato da blocchi anch'essi di 512 byte. Si considerino le tre strategie di memorizzazione del file: contigua, concatenata e indicizzata. Si supponga che le informazioni che riguardano il file (i-node), in ognuno dei tre casi, siano già in memoria centrale. L'ultimo record letto dal file è il record 3, il prossimo record da leggere è il record 8. Per ognuno dei tre casi, si calcoli quanti accessi a disco sono necessari per la lettura del record 8 e si motivi la risposta.

Esercizio 114)

Un disco magnetico ha 40 tracce, un'operazione di ricerca richiede 6 ms per lo spostamento tra una traccia e l'altra, la latenza rotazionale media è di 10 ms, il tempo di trasferimento è 25 ms per blocco.
Quanto tempo è necessario per leggere un file di 20 blocchi, se i blocchi distano mediamente 13 tracce l'uno dall'altro? Supponete che anche per accedere al primo blocco sia necessario uno spostamento della testina di 13 tracce.

Esercizio 115)

Dati i seguenti tre processi, un sistema real time è schedulabile con EDF? Motivate la risposta.

	P1	P2	P3
C (secondi di tempo di CPU)	6	5	1
P (periodo)	9	15	5

Esercizio 116)

Abbiamo due tipi di processi: tipo A e tipo B.
Tutti i processi di tipo A eseguono:
P(X)
V(Y)

Mentre i processi di tipo B eseguono:
P(Y)
P(Y)
V(X)
V(Y)

Semaforo X inizializzato a 2, Y a 0.
Supponiamo di avere, in esecuzione, tre processi di tipo A e due di tipo B.
È possibile che si ottenga una sequenza AABBA? Motivate la risposta.
PS P=Wait; V=Signal

Esercizio 117)

Considerate il seguente pseudo code che implementa un modello produttore - consumatore. Il codice è corretto? Può avvenire un deadlock oppure no? Motivate la risposta con esempi.

```
Producer () {
     mutex.P();
     emptyBuffers.P();
     put 1 coke in machine;
     fullBuffers.V();
     mutex.V();
}
```

```
Consumer () {
     mutex.P();
     fullBuffers.P();
     take 1 coke from machine;
     emptyBuffers.V();
     mutex.V();
}
```

Esercizio 118)

Si supponga di avere 4 dischi da 4 GB, tali che:
Mean time to failure = 1000 giorni;
Mean time to repair = 10 giorni.
Si calcoli la probabilità di perdere dei dati (probabilità di guasto) con RAID livello 0 e livello 1.

Esercizio 119)

Davide, Nicola, Stefano piantano semi continuamente. Davide scava i buchi. Nicola quindi mette un seme in ogni buca. Stefano poi riempie il buco.
1. Nicola non può piantare un seme a meno che non esista almeno un buco vuoto.
2. Stefano non può riempire un buco a meno che non esista almeno un buco in cui Nicola ha piantato un seme.
3. Davide deve aspettare se ci sono MAX buchi non riempiti.
4. C'è solo una pala con cui sia Davide che Stefano devono scavare e riempire i buchi, rispettivamente.
Scrivi pseudocodice per i 3 processi che rappresentano Davide, Nicola e Stefano; usate i semafori come meccanismo di sincronizzazione.
Disponete dei seguenti semafori inizializzati nel seguente modo:
semaforo Pala = 1; Semaforo “non riempito” = MAX; Semaforo seme = 0; semaforo Riempimento = 0;

Esercizio 120)

Un sistema di calcolo dispone di 30 unità a nastro magnetico e 15 stampanti.
Supponendo the tutti gli utenti utilizzino, in modo esclusivo, un nastro magnetico per leggere i dati e poi una stampante per scriverli, gestire la concorrenza usando esattamente due semafori e implementare la funzione utente().

Esercizio 121)

Dato un disco rigido con le seguenti caratteristiche:

- Capacità 512 GB
- 4 piatti / 8 facce
- 524288 tracce per faccia e 1024 settori per traccia
- Velocità di rotazione 10 000 rpm;
- Tempo medio di posizionamento della testina 1,4 ms.

Si calcoli il tempo medio di trasferimento di 32 KB supponendo che i byte siano in settori contigui di un cilindro (dunque lettura in parallelo nelle facce)

PS
512 GB = 2^{39} B
524288 = 2^{19}
1000/(10000/60)=6
10000/60 ~= 166

Esercizio 122)

Si consideri un'applicazione industriale che richieda almeno 1 TB di capacità di archiviazione e poter eseguire 4900 operazioni I/O al secondo. La dimensione del I/O dell'applicazione è 4 KB. Inoltre avete come vincolo che utilizzo del controller del disco sia inferiore o uguale al 70% (i.e. in ogni secondo potete utilizzare al massimo 0,7 secondi per effettuare operazioni di I/O).
Avete a disposizione degli hard disks con le seguenti caratteristiche:
capacità = 73 GB
Tempo di latenza = 2 ms
Tempo di ricerca medio (seek) = 5 ms
Velocità di trasferimento = 40 MB/sec
Calcolare il numero di dischi necessari.

Esercizio 123)

Dato il seguente file system:

- Dimensione del blocco=1024 byte
- Dimensione dell'indirizzo di blocco= 2 byte
- schema di allocazione indicizzato mediante 10 blocchi diretti; 1 indiretto di primo livello; 1 indiretto di 2° livello; 1 indiretto di 3° livello

Il byte 300 000 di un file si trova in quale blocco dati (un blocco dati diretto, indiretto, doppiamente indiretto o triplamente indiretto)? Motivare la risposta mostrando i calcoli effettuati.

Esercizio 124)

Considerate la seguente tabella

Processi	Tempo Arrivo	Utilizzo CPU	Coda
P1	0	4	1
P2	0	3	1
P3	0	8	2
P4	10	5	1

La priorità della coda 1 è maggiore della coda 2; la coda 1 utilizza Round Robin (quanto di tempo = 2) e la coda 2 utilizza FCFS con prelazione.
Se ci sono due processi P_a e P_b con la stessa priorità allora va in esecuzione P_a se a<b altrimenti P_b
Come hardware abbiamo a disposizione una singola CPU con un singolo core.
Mostrare ordine esecuzione processi, spiegando la vostra soluzione.

Prelazione si riferisce ad intero sistema, i.e. se arriva processo in coda 1, allora, "prende" il core se processo di coda 2 è in esecuzione.

Esercizio 125)

Una variabile condivisa x, inizializzata a zero, viene utilizzata da quattro processi W, X, Y, Z. Accesso a x è gestito mediante un semaforo S inizializzato a due. Qual è il valore massimo possibile di x dopo che tutti i processi hanno completato l'esecuzione?

Processo W	Processo X	Processo Y	Processo Z
Wait(S)	Wait(S)	Wait(S)	Wait(S)
Read(x)	Read(x)	Read(x)	Read(x)
x=x+1	x=x+1	x=x-2	x=x-2
Write(x)	Write(x)	Write(x)	Write(x)
Signal(S)	Signal(S)	Signal(S)	Signal(S)

Esercizio 126)

Dato il seguente codice, può avvenire un deadlock? In tal caso, per quali valori di (x,y,z)? Motivare la risposta.

```
Int x=0; y=0; z=0;
Sem lock1=1; lock2=1

Process foo{                    process bar {
z=z+2;                          P(lock2);
P(lock1);                       y=y+1;
x=x+2;                          P(lock1);
P(lock2);                       x=x+1;
V(lock1);                       V(lock1);
y=y+2;                          V(lock2);
V(lock2);                       z=z+1;
}                               }
```

Esercizio 127)

Dato il seguente sistema (in stato sicuro), determinare se assegnazione di 6 istanze di C a P3 garantisce il mantenimento dello stato sicuro.

4 Processi: P1...P4; 3 tipi di risorse: A (5 istanze); B (8 istanze); C (16 istanze)

	Assegnate			Massimo		
	A	B	C	A	B	C
P1	0	1	4	4	1	4
P2	2	0	1	3	1	4
P3	1	2	1	5	7	13
P4	1	0	3	1	1	6

Esercizio 128)

Abbiamo un sistema basato su tabella delle pagine in memoria centrale, inoltre esiste una TLB.
Sappiamo che:

- Ogni singolo riferimento alla memoria (e.g. accedere all'elemento oppure alla tabella delle pagine) richiede 200 ns;
- Tempo di reperimento nella TLB=20 ns.

Quale dovrebbe essere hit ratio della TLB per avere un tempo di accesso effettivo medio di 240 ns?

Esercizio 129)

Considerate una CPU quad core, abbiamo 3 processi (P0, P1 e P2) con tempo di esecuzione (nel core della data CPU) di 5, 10 e 20 ms, rispettivamente. Specificare, motivando la risposta, il tempo di esecuzione nel caso peggiore o nel caso migliore, supponendo che tutti i processi siano 100% CPU bound e non migrino dal core dopo aver iniziato esecuzione. PS: ogni singolo processo è un flusso sequenziale non parallelizzabile.

Esercizio 130)

Si determini la dimensione minima, in bit, di una FAT che gestisca al massimo una partizione da 200 GB basata su blocchi di 4 KB.

Esercizio 131)

Date due memorie cache:
A) 512 KB - hit ratio (h) 0.93
B) 1 MB - hit ratio (h) 0.97
Sapendo che, per una data cache, $S_c = \frac{T_m}{hT_c + (1-h)T_m}$

E che $T_c/T_m=0.12$, dove T_c è il cycle time della cache (entrambe) e T_m è il cycle time della memoria.
Qual è il miglioramento di performance, in percentuale, (considerando indicatore Sc) passando dalla memoria a) a quella b)?

L'indicatore S_c rappresenta **lo speedup (fattore di accelerazione) dovuto alla cache rispetto all'accesso diretto alla memoria principale**.

Formalmente:

$$S_c = \frac{T_m}{T_{medio}}$$

dove:

$$T_{medio} = hT_c + (1-h)T_m$$

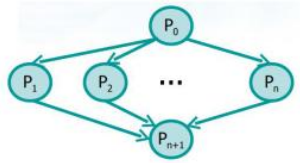
è il **tempo medio di accesso** (Average Memory Access Time, AMAT).

Il cycle time della CPU è il periodo del clock del processore (tempo tra due fronti attivi consecutivi), che determina la frequenza operativa e la granularità temporale delle istruzioni.

Il cycle time di cache e memoria è invece il tempo minimo tra due accessi consecutivi allo stesso livello di memoria: tipicamente la cache ha un cycle time molto vicino a quello della CPU, mentre la RAM ha un cycle time significativamente maggiore (latenza e accesso più lenti).

Esercizio 132)

Scrivere pseudo-codice dei processi P_0, P_i e P_{n+1} in modo da ottenere il seguente grafo delle precedenze, potete usare due semafori entrambi inizializzati a 0.




```
init (S1, 0);
init (S2, 0);
```

$P_{(n+1)}$ può essere eseguito solo dopo che P_1....P_n siano eseguiti. P_0 può essere già eseguito, non ha necessita di altri processi

Esercizio 133)

Quattro Processi, P1-P4, richiedono 4 risorse R1-R4. Si supponga che le risorse vengano allocate ad un ad una e che nessuna delle risorse venga rilasciata da un processo prima della sua terminazione. In un certo istante la fotografia del sistema è rappresentata dalle tabelle:

risorse totali

R1	R2	R3	R4
4	6	5	7

allocazione

R1	R2	R3	R4
1	0	2	2
1	2	0	1
2	1	2	0
0	0	0	2

richieste

R1	R2	R3	R4
0	1	0	2
0	4	1	0
0	2	0	5
1	2	1	0

esiste una strategia di allocazione che evita il deadlock?

Esercizio 135)

Un calcolatore ha uno spazio di indirizzamento logico per ogni processo di 128 KB, divisi in pagine di 16 KB. Se un programma ha il codice di 35250 B, un'area dati di 68000 B e uno stack di 17000 B, può essere eseguito in memoria senza ricorrere a paginazione su disco? Qual è la dimensione massima delle pagine che consente al programma di essere eseguito rimanendo completamente in memoria?

Esercizio 136)

```
1  int x = 0, y = 0, z = 0;
2  sem lock1 = 1, lock2 = 1;
3
4  process foo {          process bar {
5    z := z + 2;            P(lock2);
6    P(lock1);              y := y + 1;
7    x := x + 2;            P(lock1);
8    P(lock2);              x := x + 1;
9    V(lock1);              V(lock1);
10   y := y + 2;            V(lock2);
11   V(lock2);              z := z + 1;
12 }                      }
```

Nel dato codice c'è il rischio di deadlock (x,y,z=2,1,2), quali sono i possibili valori di (x,y,z) nel caso in cui il codice giri senza deadlock? Motivare il risultato

Esercizio 134)

Un programma in formato eseguibile è diviso in un codice di 200 KB, un'area dati di 126 KB e uno stack di 52 KB. Si ipotizzi che la macchina fisica possa indirizzare un massimo di 64 MB di memoria, che l'indirizzamento logico sia a 20 bit e che il programma sia tutto in memoria durante l'esecuzione. La memoria è organizzata a pagine di 16 KB.

Quanto è lunga la tabella delle pagine per il programma?

Quanti bit ha ogni suo elemento?

Qual è il numero di pagine logiche del più lungo programma eseguibile?

→ codice, dati e stack allocano le pagine autonomamente

Esercizio 137)

Abbiamo un computer con 1GB di memoria, il sistema operativo occupa 512 MB e i processi mediamente 128 MB, se l'attesa media dell'I/O è del 80% (p=0.8) qual è utilizzo della CPU? Aggiungendo 512 MB Ram quanto sarà nuovo utilizzo CPU? Motivare il risultato

- Il tempo di attesa del 80% è riferito all'attesa di completamento di operazioni di I/O.
- Se abbiamo n processi in memoria e il tempo di attesa medio è p, allora la probabilità che tutti i processi siano in attesa (e quindi la CPU sia intattiva) è p^n.
- L'utilizzo della CPU è, dunque, $1 - p^n$.

Esercizio 138)

Supponendo di dover valutare la fattibilità di un sistema soft real-time con eventi periodici $P_0 = 500\mu s$, $P_1 = 300\mu s$, $P_2 = 15ms$, $P_3 = 800ms$.
Con rispettivi tempi di elaborazione $C_0 = 100\mu s$, $C_1 = 100\mu s$, $C_2 = 3ms$, $C_3 = 200ms$.
Il sistema è sostenibile? Se si aggiunge un nuovo evento periodico $P_4 =$ 4 ms con $C_4 = 2ms$. Il sistema rimane sostenibile? Perché?

Esercizio 139)

Nell'ambito della gestione della memoria con liste concatenate, considerando una lista parzialmente piena di questo tipo (per ogni tripla, il primo elemento è un flag per capire se si tratta di un buco o un processo, il secondo è l'indirizzo di partenza e il terzo è la lunghezza dell'elemento): P,0,6, → H,6,3, → P,9,8 → P,17,4, →H,21,2, → P,23,6, → H,29,4,X.

- Lettera **P** rappresenta un processo.
- Lettera **H** rappresenta un "buco" di un certo numero di blocchi.

Dove verrebbe posizionato un nuovo processo che occupa 2 blocchi, usando approccio first fit e best fit.

Esercizio 140)

Assumiamo che: ogni cache miss "costi" (miss penalty) 50 cicli di clock; tutte le istruzioni impieghino due cicli di clock; il miss rate della cache sia del 2%. In media ci sono 1,33 accessi alla memoria per ogni istruzione.
Quanto aumenta performance nell'usare la data cache, rispetto a non usarla?
Performance= numero di istruzioni × (numero cicli per istruzione + accessi alla memoria per ogni istruzione × cache miss rate × cache miss penalty) × tempo di un ciclo

Esercizio 141)

Un programma in formato eseguibile è diviso in un codice di 200 KB, un'area dati di 126 KB e uno stack di 52 KB. Si ipotizzi che la macchina fisica possa indirizzare un massimo di 64 MB di memoria, che l'indirizzamento logico sia a 20 bit e che il programma sia tutto in memoria durante l'esecuzione.

Supponete che la memoria sia organizzata a segmenti lunghi al massimo 64 KB.
Quanti bit deve avere ogni registro (elemento della tabella dei segmenti) di segmentazione? Qual è il numero di registri di segmentazione del più lungo programma eseguibile?

Esercizio 142)

Un distributore di benzina ha N pompe e 1 serbatoio della capacita' di M litri. Ogni automobile all'arrivo richiede una specifica quantita' di benzina. Il serbatoio e' rifornito da una autobotte che lo riempie fino alla capacita' massima e solo se nessuna automobile sta facendo rifornimento.

le automobili possono fare benzina solo se c'e' una pompa libera, se la quantita' di benzina richiesta e' disponibile e se l'autobotte non sta riempiendo il serbatoio

```
sem distributore  // semaforo inizializzato a 1
int occupate = 0 // intero condiviso inizializzato a 0
sem serbatoio // semaforo inizializzato a 1
int disponibile = M // intero condiviso inizializzato a M

void autobotte ( )
wait (distributore)
if ( occupate > 0)
   signal (distributore)
else
   riempi ( )
   disponibile = M
   signal (distributore)
endif
end autobotte
```

Implementate automobile(), partendo dalla implementazione di autobotte(), potete usare solo semafori e variabili condivise sopracitate.

Esercizio 143)

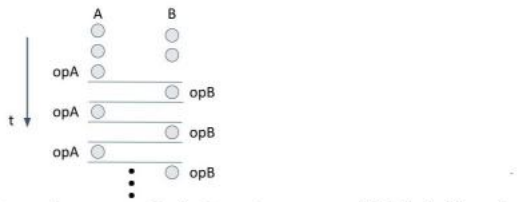
Considerate i due threads *s* e *t,* inoltre abbiamo due variabili condivise n1 e n2. La funzione unknown() ha in input due interi e ne restituisce, a random, uno dei due. Qual è il numero massimo di iterazioni dei cicli while eseguiti in s() e t()? Motivare la risposta.

```
int n1 = 0;
int n2 = 0;
```

s		t	
s_1:	**while**(n1 <= 500) {	t_1:	**while**(n2<=500) {
s_2:	n1 = n1 + 1;	t_2:	n2 = n2 + 1;
s_3:	n2 = unknown(n1,n2)		}
	}		

Esercizio 144)

- Si supponga di avere due thread A e B in esecuzione concorrente
- Si supponga, inoltre, che A e B debbano svolgere le operazioni opA e opB a turno, con A ad iniziare prima (si veda la figura sottostante)



Usare due semafori, S_1 e S_2, entrambi inizializzati a 0, per gestire tale sincronizzazione.

Esercizio 145)

Dato un programma che accede ad un singolo dispostivo di I/O, comparate la modalità di accesso con buffer alla modalità senza buffer. Dimostrate matematicamente che:

Il guadagno massimo ottenibile è un dimezzamento dei tempi d'esecuzione,

Sia C il tempo di computazione e T il tempo di lettura dati.

Il tempo d'esecuzione senza buffer (SB) è dato dalla somma del tempo di lettura e del tempo di computazione: $SB = T + C$.

Il tempo d'esecuzione con buffer (B) è invece dato da: $B = \max\{T, C\} + M$, dove M è il tempo di trasferimento dei dati dal buffer allo spazio del processo. Essendo $M << T$ possiamo approssimare $M = 0$.

Ricordate che avete 3 casi C=T; C>T; C<T.

Esercizio 146)

Dato il seguente codice:

```
1  int free := U;
2  sem enter := 1, delay := 0;
3  int cnt = 0;              # pending requests
4
5  request() { #< await (free > 0) free = free - 1; >
6
7    P(enter);
8    if (free = 0) {         # No units left
9      cnt := cnt + 1;       # one more request pending
10     V(enter);             # Release CS
11     P(delay);             # Wait to units are released
12     cnt := cnt - 1;
13   }
14   free := free - 1;       # Take one unit
15   if (free > 0 and cnt > 0) { # More than one unit was released
16     V(delay);
17   }
18   else V(enter);          # No units left or no waiting users
19 }
20
21
22 release(int number) { #< free = free + number; >
```

Implementate la funzione release(), serve per aumentare il numero di risorse “free” di un valore “number”.

Esercizio 147)

Supponete che il sistema possegga:
T blocchi totali;
U blocchi usati.
Ogni blocco viene memorizzato, nella lista dei blocchi liberi, con S bit.
Fissati S e T, può esistere un valore di U per il quale l’occupazione di memoria della bitmap, per la gestione dei blocchi liberi, sia superiore all’occupazione di memoria della lista dei blocchi liberi? Se esiste, calcolate tale valore in maniera formale.

Esercizio 148)

Si supponga di avere 4 dischi da 4 GB, tali che:
Mean time to failure = 1000 giorni;
Mean time to repair = 10 giorni.
Si calcoli la probabilità di perdere dei dati (probabilità di guasto) con RAID livello 5.

Esercizio 149)

Abbiamo un sistema basato sulla tabella delle pagine memorizzata in memoria principale, il tempo di un singolo accesso alla memoria è di 200 ns, abbiamo memoria associativa con tempo di accesso che possiamo "semplificare" a zero. Hit ratio della memoria associativa è 0.75.
Qual è l'effettivo tempo medio di accesso ad una parola presente nella memoria, partendo dall'indirizzo logico?

Esercizio 150)

Si supponga di avere una tabella delle pagine, memorizzata nei registri hardware (dunque possiamo semplificare tempo di accesso alla tabella a zero), con le seguenti caratteristiche:
8 ms per gestire page fault, 20 ms se la pagina da rimuovere è stata modificata (tali tempi comprendono anche accesso alla memoria per lettura del dato richiesto).
Inoltre abbiamo memoria con tempo di accesso di 100 ns.
Supponendo che il 70% dei page fault riguardino pagine modificate, qual è il page fault rate per ottenere un tempo di accesso medio inferiore a 200 ns? Mostrate i calcoli.

Esercizio 151)

Dato un sistema con 10 processi I/O bound e un processo CPU bound. Presupponete che i processi I/O bound eseguano una operazione di I/O dopo un millisecondo di utilizzo della CPU e che tale operazione di I/O impieghi 10 ms, mentre il processo CPU bound utilizza la CPU per 10 millisecondi. Assumete che overhead del context switching sia 0,1 ms e che si stia usando CPU a singolo core. Qual è la percentuale di tempo di utilizzo della CPU con sistema round robin e quanto di tempo:
A) 1 millisecondo.
B) 10 millisecondi.
Motivare la risposta.

PS considerate il caso peggiore considerando arrivo dei processi nella coda pronti

Esercizio 152)

Considerate un sistema di scheduling con prelazione a priorità; la priorità è dinamica (alto valore di priorità implica alta priorità del processo), quando un processo è nella coda pronti la sua priorità cambia di un valore α ogni quanto di tempo, se invece il processo è in esecuzione allora cambia di valore β per ogni quanto di tempo. A tutti i processi viene assegnata una priorità di zero quando entrano nella coda pronti. Per semplificare stiamo lavorando con CPU a singolo core.
Se $\beta>\alpha>0$ che algoritmo di scheduling abbiamo ottenuto?
Se $\alpha<\beta<0$ che algoritmo di scheduling abbiamo ottenuto?
Motivare la risposta.

notate che quando un processo entra nella coda pronti ha priorità di 0, 0 è maggiore di un numero negativo...

Esercizio 153)

Un file deve essere condiviso tra diversi processi, ognuno dei quali ha un numero unico (my_num). Il file può essere consultato simultaneamente da diversi processi, nel rispetto del seguente vincolo: la somma di tutti i numeri univoci associati a tutti i processi che stanno attualmente accedendo al file deve essere inferiore a *n*. Completate il seguente codice:

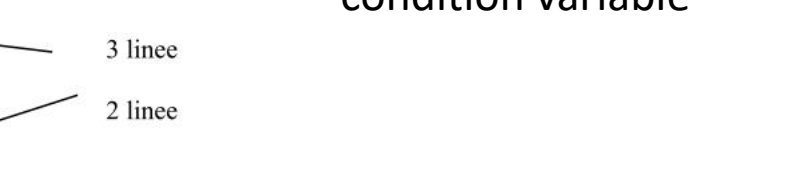
```
monitor file_access {
int curr_sum = 0;
int n;
condition c;

void access_file(int my_num) {

}

void finish_access(int my_num) {


}
}
```



con istruzione c.broadcast potete svegliare tutti i processi sospesi sulla data condition variable

Esercizio 154)

Consideriamo un sistema basato su paging, sia il tempo di accesso alla memoria che di trasferimento (memoria secondaria -> memoria principale) è pari a 25 millisecondi, indirizzi sono ottenuti mediante tabella delle pagine in memoria principale, tempo di accesso alla memoria principale pari a un microsecondo. Per migliorare performance del sistema è stata aggiunta memoria associativa (tempo di accesso approssimato a 0). Assumiamo che: 80% degli accessi si effettui usando memoria associativa, mentre il 10% degli accessi causa un page fault nella tabella delle pagine. Qual è tempo di accesso medio a tale sistema? Mostrate tutti i calcoli.

Esercizio 155)

Può un sistema RAID1 ottenere performance in lettura migliore di un sistema RAID0? Motivare la risposta

Esercizio 156)

Ogni cliente entra nel negozio e prende il suo ticket (numero incrementale, dunque diverso per ogni cliente), i venditori (un numero *n*) servono i clienti in base all'ordine di arrivo. Completate il seguente codice:

```
class CandyShop {
   public:
     void Enter(); void Finish();
   private:
     int ticket;
     Semaphore mutex;    // control access to shared variables
     Semaphore servers; // Sales people
}
CandyShop::CandyShop(int n) {
     ticket = 0;
     mutex.value = 1;
     servers.value = n;  // Number of Sales people
}
CandyShop::Enter() {
```



4 linee

1 linea



```
}
CandyShop:: Finish() {

}
```

Esercizio 157)

Dato un sistema con 6 dischi (a,b,c,d,e,f) con *n* thread in competizione per accedere ad essi. Ogni thread potrebbe chiedere fino a due dischi, qual è il valore di massimo di *n* affinchè nel sistema non possa avvenire deadlock? Perchè?

si intende che per essere eseguito un thread necessiti di uno o due dischi, ogni disco è una risorsa

Esercizio 158)

Scrivere un monitor che implementa clock che permetta al programma di ritardare l'esecuzione di se stesso di un numero predefinito di unità temporali (ticks).
Usate funzione read_clock() per leggere clock hardware, tale clock hardware invoca una funzione del monitor a intervalli regolari, tale funzione è:
Void tick() {
c.broadcast()
}
Implementate una funzione delay(int ticks) per gestire sospensione del processo, i.e. invocando tale funzione il processo si sospende per ticks unità temporali.
Avete a disposizione una sola variabile condition

```
monitor alarm {
    condition c;
```

Esercizio 159)

Implementate la funzione decrease_count(int count), un processo chiama decrease_count(count) per ottenere count risorse, se non disponibili si sospende altrimenti ridurre di count le risorse disponibili. Implementate inoltre la funzione increase_count(count) per aumentare il numero di risorse disponibili di count elementi.
Avete a disposizione il seguente monitor, ricordate di gestire il "risveglio" dei processi sospesi:

```
monitor resources
{
    int available_resources;
    condition resources_avail;

    int decrease_count(int count)
    {



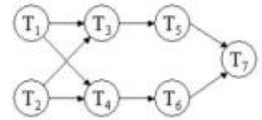
    }

    int increase_count(int count)
    {


    }
```

Esercizio 162)

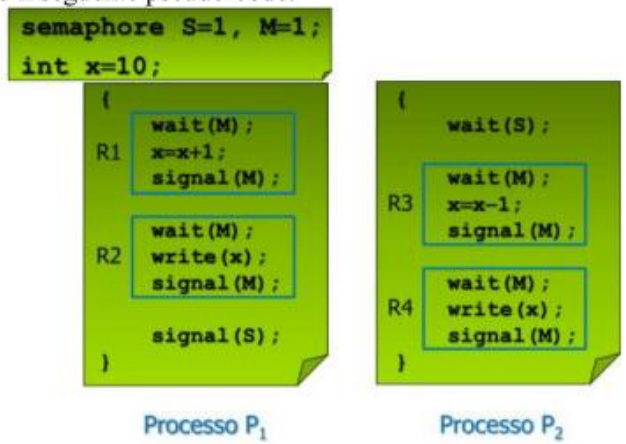
Si consideri il seguente grafo di precedenza fra processi:



Usate 4 semafori per ottenere tali precedenze, riportate pseudo code per ottenere tale grafo.

Esercizio 163)

Dato il seguente pseudo-code:



Determinare possibili output dell'esecuzione parallela di P_1 e P_2 in CPU a singolo core.

ogni entry della tabella delle pagine è 32 bit

Esercizio 160)

Supponiamo di utilizzare la seguente variante della tabella delle pagine su 2 livelli: la prima metà degli elementi della tabella indicizzano direttamente pagine fisiche, la seconda metà indicizza tabelle della pagine di 2° livello (come in normale sistema su 2 livelli).
Inoltre abbiamo indirizzi a 32 bit, pagine di 4 KB (i.e. 2^{12}B) e ogni tabella delle pagine (di entrambi i livelli) occupa una pagina.
Qual è la dimensione dello spazio di indirizzamento, in byte, di tale sistema? Motivate il risultato. PS Potete lasciare il risultato nella forma 2^x+2^y

Esercizio 161)

State utilizzando hamming code per trovare errore in sequenza di bit, avete sequenze di 6 bit (in totale, sia parità che dati), si utilizzano bit di parità dispari, dunque: si pone tale bit uguale a 1 se il numero di "1" in un certo insieme di bit è pari (facendo diventare il numero totale di "1", incluso il bit di parità, dispari).
Data la sequenza:
0 0 1 1 1 0
Correggete eventuale errore usando il metodo basato sul codice di Hamming. N.B. algoritmo funziona come nelle dispense, unica differenza che nell'esempio delle dispense i bit di parità erano pari ora dispari.

Esercizio 164)

Si sta utilizzando un file system basato su FAT-32 e l'overhead totale di ogni voce della FAT è di 4 byte (spazio occupato in ogni elemento della FAT, oltre che i dati veri e propri). Dato un disco di 100 MB su cui è memorizzato il file system e la dimensione del blocco dati sia di 1000 byte. Qual è la dimensione massima di un file che può essere memorizzato su questo disco? Perchè?

Esercizio 165)

Considerate il seguente pseudo-code:

```
const int n = 50;
int tally;
void total()
{
   int count;
   for (count = 1; count <= n; count++){
      tally++;
   }
}
void main()
{
   tally = 0;
   parbegin (total (), total ());
   write (tally);
}
```

Tally è una variabile condivisa, qual è il valore minimo e massimo che può assumere? Motivate il risultato. Istruzione parbegin() lancia in parallelo due funzioni total(). PS tally++ NON è operazione atomica

Esercizio 166)

Abbiamo un sistema con memoria virtuale, indirizzi a 32 bit e pagine da 1Kbyte. Ogni entry della tabella delle pagine è di 32 bit (di cui una parte saranno bit di informazione, come il dirty bit), inoltre, ogni tabella delle pagine occupa al più una singola pagina, i.e. non possiamo avere tabella delle pagine di dimensione maggiore di un 1KB.
Quanti livelli della tabella delle pagine sono necessari per gestire tale memoria? Motivare la risposta. Ricordate, avendo tabella delle pagine di 1 KB, non è detto che una tabella delle pagine riesca a contenere tutte le possibili entry, dunque potrebbe servire un sistema gerarchico.

Esercizio 167)

Dato il seguente pseudo-code:

```
semaphore S=2, T=0;
int x=0;
```

Processo P_1

```
{
    wait(T);
    x=5;
    signal(S);
    x=6;
}
```

Processo P_2

```
{
    wait(S);
    x=1;
    signal(T);
    wait(S);
    x=10;
    write(x);
}
```

Determinare possibili output dell'esecuzione parallela di P_1 e P_2 in CPU a singolo core.

Il risultato è diverso in base a come considerate "possibili output", nel senso se consideriamo il valore scritto da write(x), oppure quali sono i valori finali di x dopo l'esecuzione dei due processi

Esercizio 168)

Un torrente è attraversato da un ponte. Il ponte è lungo quanto tre automobili, ma è largo quanto un'automobile, quindi la circolazione è a senso unico alternato.
Le regole sono:
1. indipendentemente dalla sponda da cui provengono, le auto entrano nel ponte nell'ordine di arrivo;
2. un'auto non può accedere al ponte se questo è occupato da una o più auto provenienti dalla sponda opposta;
3. un'auto non può accedere al ponte se questo è già occupato da tre auto.

Scrivete funzioni corrispondenti alle macchine provenienti dalla riva destra e dalla riva sinistra.
Partite dal seguente code, funzione macchina_dalla_riva_sinistra() è solo iniziata, dovete completarla e poi scrivere la versione per riva destra. Non potete aggiungere altri semafori o variabili globali.

```
semaphore accesso = 1; /* forza l'ordinamento delle macchine che arrivano */
semaphore portata = 3; /* capacita' del ponte */
semaphore mutexsx = 1; /* per controllo di flusso e mutex */
semaphore mutexdx = 1; /* per controllo di flusso e mutex */
int nsx = 0; /* macchine che hanno ottenuto accesso da sx */
int ndx = 0; /* macchine che hanno ottenuto accesso da dx */

macchina_dalla_riva_sinistra() {
  wait(accesso); /* inizio protocollo di accesso */
  wait(mutexsx); /* per evitare scontri e per mutex su nsx e ndx */
  ++nsx;
  if (nsx==1) /* prima di un gruppo in questa direzione */
     wait(mutexdx); /* blocca macchine dall'altra riva */
```

PS ++nsx è operatore di incremento prefisso, valore della variabile viene prima incrementato di 1, poi vengono (eventualmente) eseguiti ulteriori calcoli.

Esercizio 169)

Esiste caso in cui il seguente code non gestisca adeguatamente accesso alla sezione critica? PS con /* remainder */ intendete altre funzioni non specificate, per le quali è necessario gestire concorrenza, dunque si trovano nella sezione critica.
Partite da: turn ha valore 0; parte esecuzione di P(1); P(1) imposta blocked[1] a true... continuate voi, accesso a sezione critica è sempre eseguito correttamente?
PS parbegin() esegue P(0) e P(1) in parallelo.

```
boolean blocked [2];
int turn;
void P (int id)
{
 while (true) {
    blocked[id] = true;
    while (turn != id) {
       while (blocked[1-id]) = = false;
          /* do nothing */;
       turn = id;
    }
    /* critical section */
    blocked[id] = false;
    /* remainder */
 }
}
void main()
{
 blocked[0] = false;
 blocked[1] = false;
 turn = 0;
 parbegin (P(0), P(1));
}
```

Esercizio 170)

Data applicazione con una parte del code non parallelizzabile, pari al 40% del tempo di esecuzione, se la restante parte non ha upper bound al numero di core che può utilizzare in parallelo, qual è aumento di velocità di esecuzione di tale applicazione se eseguita su un ipotetico cluster dotato di un numero infinito di core?

Esercizio 171)

Dati due task periodici: *x* con CPU-burst di 2 ms e periodo di 5 ms; *y* con CPU-burst di 4 ms e periodo di 7 ms. Si utilizzi EDF per scheduling, mostrare il diagramma di Gantt e commentatelo, vengono rispettate deadline per gestire in maniera real time tali processi?

Esercizio 172)

- Si considerino due processi, un produttore P ed un consumatore C, ed un buffer con capacità pari ad 1
- Sia

```
semaphore empty=1;
```

l'unica variabile condivisa (oltre al buffer)

- Si considerino le seguenti porzioni di codice includenti la sezione critica per l'accesso al buffer da parte di P e C

Processo P

```
   while(true) {
p1     wait(empty);
p2     enter_item(item);
   }
```

Processo C

```
   while(true) {
c1     remove_item(item);
c2     signal(empty);
   }
```

Supponendo che per prime vengano eseguite p1 e p2, esiste sequenza di istruzioni non corretta per il formalismo produttore-consumatore? E.g. produttore aggiunge a buffer pieno, consumatore legge da buffer vuoto... Oppure codice è corretto? Motivare risposta.

Esercizio 173)

- Si consideri la situazione in cui vi siano in esecuzione quattro thread, T_A, T_B, T_C e T_D, con le matrici relative alle risorse allocate ed al numero massimo di risorse richieste dai thread definite come segue:

$$\text{Allocation} = \begin{pmatrix} 1\,0\,2\,1\,1 \\ 2\,0\,1\,1\,1 \\ 1\,1\,0\,1\,0 \\ 1\,1\,1\,1\,0 \end{pmatrix} \qquad \text{Max} = \begin{pmatrix} 1\,1\,2\,1\,3 \\ 2\,2\,2\,1\,1 \\ 2\,1\,3\,1\,0 \\ 1\,1\,2\,2\,1 \end{pmatrix}$$

- Se il vettore delle risorse disponibili è

$$\text{Available} = (0,0,x,1,1),$$

qual è il minimo valore di x che rende lo stato attuale sicuro?

Esercizio 174)

Si consideri uno spazio degli indirizzi logici costituito da 128 pagine da 2KB, mappato su una memoria fisica di 64 frame.

- Qual è la dimensione della tabella delle pagine invertita supponendo che il sistema non supporti l'esecuzione contemporanea di più di 512 processi alla volta?

Esercizio 175)

Si consideri il seguente frammento di programma in linguaggio C:

```
{
  ...
  int a[1000], b[1000];

  for (i=0; i<999; i++)
      a[i]=a[i]+b[i];
  ...
}
```

Si supponga che un intero occupi una parola (2 byte) e si calcoli il numero di page fault generati dal programma se si possono allocare, in ogni momento...

- ...2 pagine contenenti 100 parole per ognuno dei vettori **a** e **b**;
- ...3 pagine contenenti 100 parole per ognuno dei vettori **a** e **b**;
- ...2 pagine contenenti 500 parole per ognuno dei vettori **a** e **b**.

Si utilizzi la strategia FIFO; si assuma infine che il codice e la variabile **i** siano collocati in un'altra pagina e che nessun accesso a tali entità provochi un page fault.

In totale ci sono queste pagine, non separatamente per a e b (i.e. 2 pagine per a e due per b) ma due (o tre) pagine in tutto

Esercizio 176)

Si consideri un file system con la dimensione di blocco logico di 2KB e con indirizzi di blocchi a 16 bit. Determinare la dimensione massima di un file in caso di

- allocazione indicizzata con due livelli di blocco.

Esercizio 177)

Dati i seguenti processi, dove: S è inizializzato a 1; X è inizializzato a 5:

```
Process A          Process B
    int Y;             int Z;
    wait(S);           wait(S);
A1: Y = X*2;       B1: Z = X+1;
A2: X = Y;         B2: X = Z;
    signal(S);         signal(S);
```

Quali valori di X sono possibili dopo che ENTRAMBI i processi hanno terminato l'esecuzione?

Esercizio 178)

Dato il seguente codice:

```
#define Size 64
 int  A[Size; Size], B[Size; Size], C[Size; Size];
 int register  i, j;
 for  (j = 0; j< Size; j ++)
   for  (i = 0; i< Size; i++)
C[i; j] = A[i; j] + B[i; j];
```

$64^2*4B=16K=16$ pagine

Il sistema utilizza una paginazione su richiesta, la dimensione della pagina è 1 KB, ogni intero occupa quattro byte.
Supponete che il sistema allochi un working set di quattro pagine, una di queste pagine sarà usata dal programma, cioè dal codice, mentre tre pagine saranno usate dai dati. Per semplificare i registri *i* e *j* non vengono considerati nel calcolo dei page fault e nell'occupazione di memoria. È chiaro che ogni array A, B e C richiede uno spazio di 16 pagine. Ad esempio, A[0,0]-A[0,63], A[1,0]-A[1,63], A[2,0]-A[2,63] e A[3,0]-A[3,63] saranno memorizzati nella prima pagina di dati.
In tale codice ci sono 3 page fault ogni 4 esecuzioni di C[i;j]=A[i;j]+B[i;j].
È possibile modificare il programma in modo che ci siano tre page fault per ogni 256 esecuzioni di C[i;j]=A[i;j]+B[i;j]? In caso di risposta affermativa riportate il nuovo codice, in caso di risposta negativa motivatela.

Esercizio 179)

Un sistema dispone di tre risorse: A, B e C. Vengono eseguiti tre processi, denominati X, Y e Z, ciascuno dei quali utilizza periodicamente due delle tre risorse:

- Il processo X acquisisce A, poi B, li usa entrambi e poi li rilascia;
- Il processo Y acquisisce B, poi C, li usa entrambi e poi li rilascia;
- Il processo Z acquisisce C, poi A, li usa entrambi e poi li rilascia.

Esiste rischio deadlock con tutti e tre i processi in esecuzione? In caso affermativo modificare ordine di acquisizione di uno solo dei tre processi, in modo che non si verifichi un deadlock con tre processi in esecuzione.

Esercizio 180)

Consideriamo il seguente sistema: ogni processo può avere 1024 pagine e la tabella delle pagine relativa si trova in memoria, il tempo di accesso e lettura di un elemento nella tabella delle pagine impiega 5 nanosecondi, esiste anche una memoria associativa di 32 coppie "pagina virtuale - pagina fisica", il tempo di accesso e lettura dalla memoria associativa è di un nanosecondo, approssimate a 0 il tempo di controllo se una pagina virtuale sia presente nella memoria associativa. Qual è il valore di hit rate, sulla memoria associativa, necessario per far sì che il tempo di lettura medio dell'indirizzo di una pagina fisica, data la pagina virtuale, sia di due nanosecondi?

Esercizio 181)

Supponiamo che una data istruzione X impieghi un nanosecondo e che ogni page fault richieda *n* nanosecondi per la gestione, qual è la formula matematica per l'effettivo tempo medio di X se tale istruzione causa un page fault ogni *k* volte che viene eseguita? I.e. riportare e motivare formula, e.g. 'tempo medio' = $3n+k+1$. Ogni *k* volte che viene eseguita, e.g. se ogni 3 volte c'è un page fault e questo page fault richieda 2 nanosecondi, allora se eseguiamo 3 volte l'istruzione avremo 1+1+1+2 nanosecondi, dunque il tempo medio sarà 5/3.

Esercizio 182)

Dato il seguente, non corretto, codice:

```
sema gate     = 1
sema outgoing = 1
sema empty    = 10
sema occupied = 0
```

```
Supplier                Deliverer_X             Deliverer_Y
{                       {                       {
  while (TRUE)            while (TRUE)            while (TRUE)
  {                       {                       {
    P(gate);                P(gate);                P(gate);
    <Pass through gate>;    <Pass through gate>;    <Pass through gate>;
    V(gate);                                        V(gate);

                                                    P(outgoing);
    <Enter incoming         <Enter outgoing         <Enter outgoing
    goods section>;         goods section>;         goods section>;


                            P(occupied);
                            P(occupied);
    <Unload 3 packets>;     <Pick 2 packets>;       <Pick 1 packet>;
                            V(empty);               V(empty);
                            V(empty);


    <Leave incoming         <Leave outgoing         <Leave outgoing
    goods section>;         goods section>;         goods section>;
                            V(outgoing);

    P(gate);                P(gate);
    <Pass through gate>;    <Pass through gate>;    <Pass through gate>;
    V(gate);                V(gate);                V(gate);
    }                       }                       }
}                       }                       }
```

Modificatelo per soddisfare le seguenti condizioni:
- Solo un singolo processo può attraversare il gate.
- Solo uno dei due deliverer esistenti può accedere alla sezione "outgoing goods section", i.e. non possono contemporaneamente accedere entrambi a tale sezione.
- Il supplier e uno dei deliverer devono poter scaricare e prelevare la merce in modo simultaneo (i.e. deve essere possibile esecuzione parallela di "unload 3 packets" e "Pick *x* packets").
- La capacità del magazzino è pari a 10.
- All'inizio, il magazzino è vuoto.

Esercizio 183)

Dati i seguenti due processi, eseguiti in concorrenza, che condividono i semafori S e R, entrambi inizializzati a uno, e la variabile *x* inizializzata a zero:

```
void foo( ) {
   do {
    semWait(S);
    semWait(R);
    x++;
    semSignal(S);
    SemSignal(R);
 } while (1);
}
```

```
void bar( ) {
do {
   semWait(R);
   semWait(S);
   x--;
   semSignal(S;
   SemSignal(R);
} while (1);
}
```

Non abbiamo informazioni su priorità ed algoritmo di scheduling, un problema potrebbe essere in tale parte, i.e. sempre in esecuzione uno dei due processi. Non avendo tali informazione, presupponiamo scheduling fair

Può l'esecuzione concorrente di questi due processi risultare in un ritardo indefinito di uno dei due? I.e. uno dei due "lavora" sempre e l'altro rimane sempre sospeso, non dovete mostrare il caso in cui ci sia blocco di entrambi i processi. In caso affermativo mostrare qual è l'esecuzione che porta a un ritardo indefinito di uno dei due, e l'esecuzione dell'altro, in caso negativo motivare perché non esiste tale ritardo indefinito.

Esercizio 184)

Si utilizzi il codice di Hamming per una parola di 7 bit (totali, sia dati che parità), usiamo parità **dispari**, vincolo aggiuntivo:
i bit di dati rappresentano un numero binario, tale numero deve essere un multiplo di 5, altrimenti sequenza è considerate non corretta (e.g. 1 0 0 0 = 8 non è multiplo di 5)
La parola (ricordate comprende sia dati che parità) è
1 0 0 1 1 1 1
Correggere l'errore (se presente) e determinare se la sequenza sia corretta.

Esercizio 185)

Dato il seguente sistema:

```
semaphore U = 3;
semaphore V = 0;

[Process 1]      [Process 2]      [Process 3]

L1:wait(U)       L2:wait(V)       L3:wait(V)
   type("C")        type("A")        type("D")
   signal(V)        type("B")        goto L3
   goto L1          signal(V)
                    goto L2
```

Dopo un tempo tendente all'infinito, blocchiamo l'esecuzione del sistema: qual è il numero massimo di volte in cui viene stampata la lettera 'C' e quante volte la 'D'?

Esercizio 186)

Dato il seguente scheletro:
Variabile counter inizializzata a 10.

```
Process A                          Process B
...                                ...
A1: tempA = counter + 1;           B1: tempB = counter + 2;
A2: counter = tempA;               B2: counter = tempB;
...                                ...
```

Modificare code nella maniera più semplice possibile, in modo tale che il valore di counter sia 13 dopo esecuzione di A e B.
Potete usare un singolo semaforo sync, scegliete voi con che valore inizializzarlo. PS non potete modificare code già presente e inizializzazione di counter, potete solo aggiungere altre righe di codice legate alla gestione del semaforo sync.

Esercizio 187) opzionale GOlang

Quali sono i possibili output del seguente codice GOlang, motivando la soluzione:

```
package main
import (
        "fmt"
        "math/rand"
        "time"
)
func sendAfterDelay(num int, ch chan int) {
        delay := time.Duration(rand.Intn(4)+1) * time.Second
        time.Sleep(delay)
        ch <- num
}
func main() {
        rand.Seed(time.Now().UnixNano())
        c1 := make(chan int)
        c2 := make(chan int)
        c3 := make(chan int)
        go sendAfterDelay(10, c1)
        go sendAfterDelay(20, c2)
        go sendAfterDelay(30, c3)
```

rand.Intn(4) generates a random integer in the range [0, 3].
time.Duration(...) converts the resulting integer to a time.Duration type.
Important: A Duration is expressed in nanoseconds by default.
So time.Duration(3) equals 3ns unless scaled using time.Second

```
timeout := time.After(3 * time.Second)
received := 0
for {
        select {
        case x := <-c1:
                fmt.Println("Ricevuto da c1:", x)
                received++
        case x := <-c2:
                fmt.Println("Ricevuto da c2:", x)
                received++
        case x := <-c3:
                fmt.Println("Ricevuto da c3:", x)
                received++
        case <-timeout:
                fmt.Println("Fine: scaduto il tempo")
                return
        }

        if received == 3 {
                return
        }
}
}
```

time.After(3 * time.Second)
creates a timer channel that "fires" after 3 seconds, allowing you to pause or timeout parts of your code.

Esercizio 188)

Supponiamo che un'istruzione A impieghi *t* ns per essere eseguita in condizioni normali, durante esecuzione del programma si verificano due tipi di interruzioni:
1.Page Fault, ogni k_1 esecuzioni di A si verifica un page fault che aggiunge un overhead di n_1 ns;
2.Interruzione Hardware, ogni k_2 esecuzioni di A si verifica interruzione hardware che aggiunge overhead di n_2 ns.
Determinate la formula per il tempo medio effettivo di esecuzione di A: partite da "tempo medio effettivo"=*t*+Y+Z. quali sono i valori di Y e Z? Riportate formula chiara e semplice, motivandola. Calcolate il tempo medio per il caso specifico: t=2 ns; k_1=5; n_1=3 ns; k_2=10; n_2= 8 ns.

Esercizio 189)

Si hanno due processi, A e B, che condividono due variabili globali:
·int X, inizializzato a 3
·int W, inizializzato a 10
È presente un semaforo binario S, inizializzato a 1. Di seguito sono mostrate le istruzioni di ciascun processo:

Processo A
```
int Y;
// Prima sezione critica
wait(S);
  Y = X + 5;
  X = Y + 2;
signal(S);
// Seconda sezione critica
wait(S);
  W = W - X;
signal(S);
```

Processo B
```
int Z;
wait(S);
  Z = W / 2;
  W = Z;
  X = X + 1;
signal(S);
```

Domanda: Quali coppie di valori finali (X, W) possono presentarsi dopo che entrambi i processi *A* e *B* hanno terminato l'esecuzione? Motivare la risposta

Esercizio 190)

Si consideri il seguente monitor, chiamato GestoreDati, che racchiude due variabili condivise: X e W. Il monitor fornisce tre metodi pubblici:
```
monitor GestoreDati {
private:
   int X = 5;
   int W = 20;

public:
   void opA1() {
      // Operazione A1
      int Y = X + 2;
      X = Y;
   }

   void opA2() {
      // Operazione A2
      W = W - X;
   }

   void opB() {
      // Operazione B
      W = W / 2;
      X = X + 3;
   }
}
```
Abbiamo due processi, *A* e *B*, che invocano i metodi del monitor nel seguente ordine:
Processo *A* esegue in sequenza:
1.opA1()
2.opA2()
Processo *B* esegue:
1.opB()
Domanda: Quali valori finali di X e W sono possibili dopo che entrambi i processi hanno terminato l'esecuzione?

Esercizio 191)

Considerate configurazioni RAID 5, ogni disco ha una probabilità indipendente di guasto pari al 10% in un anno.
Tale configurazione utilizza 3 dischi. In un array RAID 5, il sistema tollera il guasto di un solo disco. Calcolate la probabilità di guasto del sistema in configurazione RAID 5 in un anno (non considerate la possibilità di riparare o sostituire dischi). Supponete di utilizzare distribuzione binomiale
Hint: avete più dischi, considerate tutti i possibili casi di guasto...

$$\binom{n}{k} p^k q^{n-k}$$

Esercizio 192) Opzionale GOlang

Completate il seguente codice, i.e. inserite codice nei rettangoli grigi, aggiungete commenti nel codice

```
package main
import (
"fmt"
"sync"
)

func squareNumber(in <-chan int, out chan<- int, wg *sync.WaitGroup) {
num := <-in
out <- num * num
wg.[redacted]
}

func main() {
in := make(chan int, 2)
out := make(chan int, 2)

var num1, num2 int
fmt.Print("Inserisci il primo numero: ")
fmt.Scan(&num1)
fmt.Print("Inserisci il secondo numero: ")
fmt.Scan(&num2)

in <- num1
in <- num2

var wg sync.WaitGroup
[redacted]

go squareNumber[redacted]
go squareNumber[redacted]

[redacted]
[redacted]

sum := 0
for val := range out {
[redacted]
}

fmt.Println("Somma dei quadrati:", sum)
}
```

Esercizio 193)

Supponiamo che al tempo T0 si avvii il processo P0 con durata 10 secondi (CPU bound) e a T0+3 venga originato processo P1 con durata 6 secondi (CPU bound) . Supponendo di utilizzare scheduler Shortest Remaining Time, si calcoli il tempo di completamento del processo P0 rispetto a T0, i.e. P0 termina a T0+x secondi, qual è il valore di x?

Esercizio 194)

Un sistema operativo ha gestito 4 processi con le seguenti caratteristiche:

Processo	Arrivo	CPU burst
P1	0	8
P2	1	4
P3	2	2
P4	3	1

Dopo l'esecuzione, si osservano i seguenti tempi di completamento: P1 → 15; P2 → 8; P3 → 4; P4 → 5.
Quale algoritmo di scheduling è stato usato? Con che caratteristiche? Motivare la risposta

Esercizio 195)

Dato il seguente codice, si determini, motivando la risposta, qual è il numero minimo di volte che possa venire stampato "A"

```
semaphore U = 3;
semaphore V = 0;

[Process 1]      [Process 2]     [Process 3]

L1:wait(U)       L2:wait(V)      L3:wait(V)
   type("C")        type("A")       type("D")
   signal(V)        type("B")       goto L3
   goto L1          signal(V)
                    goto L2
```

Esercizio 196)

Abbiamo 3 processi concorrenti, A, B e C che accedono alla memoria condivisa M. Periodicamente i processi A e B aggiornano, rispettivamente, la prima metà e la seconda metà di M. Quando l'aggiornamento è stato eseguito da entrambi, il processo A riscrive il contenuto dell'intero segmento M invertendo l'ordine dei byte. Il processo C legge periodicamente l'intero contenuto del segmento di memoria condivisa M; la lettura è abilitata solo dopo che l'aggiornamento da parte di A e B, e l'inversione da parte di A, siano stati effettuati. Nessun nuovo aggiornamento può avvenire su M, da parte di A e B, prima che C abbia letto il contenuto registrato in M. Si completi il seguente pseudo-codice, non potete aggiungere semafori o variabili condivise.

Semaphore SA=1;//semaforo per abilitare la scrittura del processo A
Semaphore SB=1;//semaforo per abilitare la scrittura del processo B
Semaphore SC=0;//semaforo per abilitare la lettura del processo C
Semaphore INV=0;//semaforo per abilitare inversione dei byte da parte di A

SCRIVI:
wait (SB);
<aggiornamento dati 2° metà del segmento>
[redacted]

SCRIVI-INVERTI:
wait (SA);
<aggiornamento dati 1° metà del segmento>
[redacted]
<inversione ordine byte di M>;
signal (SC);

LEGGI:
[redacted]
<leggi dati da M>;
[redacted]

Esercizio 197)

Supponiamo di usare FAT32, il file file A.txt ha come cluster iniziale il 5 e la FAT mostra:
FAT[5]=6
FAT[6]=7
FAT[7]=5
Tale situazione è corretta oppure sussiste un problema? Motivate vostra risposta, se sussiste problema proponete soluzione.

Esercizio 198)

In un sistema di memoria a due livelli (cache e RAM), il tempo medio di accesso può essere modellato come $T=p \cdot tc+(1-p) \cdot tr$
dove: *p* è la probabilità che accesso sia soddisfatto dalla cache; tc=2 ns (cache); tr=50 ns (RAM).
Dalle simulazioni, la probabilità di 'cache hit' varia nel tempo secondo $p(t)=(t/(1+t))$, considerate il tempo come variabile continua.
Trova il tempo medio di accesso medio tra t=0 e t =10. PS $48 \times \log(11)=48 \times 2.3979 \sim=115$

Esercizio 199)

Quali sono i possibili valori (motivate risposta) in output di:

```
package main
import (
   "fmt"
   "sync"
)
 func worker(id int, wg *sync.WaitGroup) {
    defer wg.Done()
    fmt.Printf("Worker %d started\n", id)
 }
 func main() {
    var wg sync.WaitGroup
    for i := 1; i <= 3; i++ {
       wg.Add(1)
       go worker(i, &wg)
    }
    wg.Wait()
    fmt.Println("All workers done")
 }
```

Esercizio 200)

Considerate sei processi P1, P2, P3, P4, P5, P6 relativi ad un dato programma; dove:
P1 precede sia P2 che P3;
P2 precede sia P4 che P5;
P3 precede sia P4 che P5;
P5 precede P6.
Utilizzare 4 semafori inizializzati a 0, in modo che le precedenze descritte siano verificate, scrivere pseudo-codice relativo ai 6 processi. N.B. non è un loop, dopo che i 6 processi sono terminati il programma termina.

Esercizio 201)

Supponiamo che un disco di 5000 cilindri possa eseguire una ricerca verso un cilindro adiacente in 1 millisecondo e una ricerca dal primo all'ultimo cilindro in 18 millisecondi.
Scrivere un'equazione per il tempo di ricerca in funzione della distanza di ricerca. Questa equazione deve essere della forma $t = x + y\sqrt{L}$, dove t è il tempo in millisecondi e L è la distanza di ricerca in cilindri, determinate i valori di x e y.
PS= 4999^0.5~70; (70-18)/69~0.754

Esercizio 202)

Un'applicazione ha tempo di esecuzione seriale su p=1 core pari a T(1)=1.0 e si compone di tre parti:
S: parte strettamente seriale, pari al 30% del tempo;
A: parte ampiamente parallelizzabile (all'infinito), pari al 40%, però la parallelizzazione introduce un overhead di $(0{,}02 \times \log_2(p)/p)$;
B: parte parallelizzabile, pari al 30%, che non scala oltre p=16 core (non c'è nessun overhead);
C: overhead di sincronizzazione globale (presente solo quando si parallelizza il codice) che incide come $0{,}05 \cdot (1-1/p)$.
Si modelli il tempo su p (p>=2) core come:
$T(p)=S+A+B+C$.
Dal testo chiaramente abbiamo S=0.3 e $C=0{,}05 \cdot (1-1/p)$; determinate A e B, poi calcolate T(2) e $T(\infty)$. Motivare i valori ottenuti. PS 0.3/16=0.01875

Esercizio 203)

Si considerino tre semafori inizializzati come segue:
U=3,V=1,W=0.
Sono in esecuzione i tre processi (L1, L2, L3) ciclici:

```
                L2: wait(V);
L1: wait(U);        type("A");
    type("C");      wait(W);
    signal(V);      type("B");  L3: wait(V);
    signal(W);      signal(V);      type("D");
    goto L1;        goto L2;        goto L3;
```

Dopo un tempo tendente infinito interrompiamo l'esecuzione: qual è il numero massimo di volte che può essere stampata la lettera "D"?

Esercizio 204)

Sono in esecuzione cinque thread (T0, T1, T2, T3, T4) e quattro risorse (R1, R2, R3, R4). Le matrici Allocation e Max sono:

$$\text{Allocation} = \begin{pmatrix} 1 & 0 & 1 & 1 \\ 2 & 1 & 0 & 0 \\ 0 & 1 & 1 & 0 \\ 1 & 2 & 0 & 1 \\ 0 & 0 & 2 & 1 \end{pmatrix} \qquad \text{Max} = \begin{pmatrix} 2 & 1 & 3 & 2 \\ 3 & 2 & 1 & 1 \\ 1 & 2 & 2 & 1 \\ 2 & 3 & 1 & 2 \\ 1 & 1 & 3 & 2 \end{pmatrix}$$

sia il vettore delle risorse disponibili uguale a (2, 2, x, 3)
Utilizzando il minimo valore di x che rende lo stato sicuro, T0 richiede (0,0,2,0), quante unità aggiuntive di R3 (rispetto alle disponibili attuali) servono al minimo per poterla concedere subito mantenendo lo stato sicuro? Motivate la risposta

Esercizio 205) Completare le righe mancanti e spiegare come i 3 worker lavorano in parallelo.

```
package main
import (
        "fmt"
        "sync"
)

func worker(id int, nums <-chan int, results chan<- int, wg *sync.WaitGroup) {
[redacted]
        for n := range nums {
                results <- n * n        }
}

func main() {
        nums := make(chan int, 5)
        results := make(chan int, 5)
        var wg sync.WaitGroup

        for i := 0; i < 3; i++ {
[redacted]
        }
        for i := 1; i <= 5; i++ {
                nums <- i
        }
[redacted]

        // stampa i risultati ricevuti
        for r := range results {
                fmt.Println(r)
        }
}
```

Esercizio 206) Usando Shortest Remaining Time First, versione con prelazione dello Shortest Job First, calcolare diagramma Gantt e tempi di turnaround dei seguenti processi, prima in un sistema a singolo core e poi in un sistema a due core, supponiamo non esistano altri processi nel sistema oltre a quelli elencati in tabella:

Processo	Arrivo (t)	Durata (Burst)
P1	0	8 ms
P2	1	4 ms
P3	2	2 ms
P4	3	1 ms

Dettagli su questa “particolare” implementazione di SRT:
PS se due processi hanno lo stesso tempo di esecuzione rimanente, allora non avviene la prelazione. Scheduler, per controllare la durata dei processi in esecuzione, esamina tutti i core e, se necessario, seleziona processo con durata maggiore (rispetto al nuovo processo) da bloccare, per portare in esecuzione nuovo processo. Fra i processi di durata maggiore al nuovo arrivato, seleziono quello relativo allo shortest job (i.e. quello con durata minore considerando il set di processi che hanno durata maggiore al nuovo processo).

Esercizio 207)

Si consideri un sistema che utilizza il **Codice di Hamming (7,4)** con parità **pari**. I bit di controllo (P) sono inseriti nelle posizioni potenza di 2 $(1, 2, 4)$.

1. **Codifica:** Generare la parola di codice per i bit dati $D = 0000$.
2. **Errore:** Si ipotizzi che durante la trasmissione si verifichino **3 errori** nelle posizioni **1, 2 e 3**.

Esercizio 208)

a)

Abbiamo un array RAID 3 con 3 dischi dati ($D1, D2, D3$) e 1 disco di parità (P).

A un certo istante, i byte salvati nella stessa posizione sui dischi sono:

- **D1:** 10101100
- **D2:** 00110011
- **D3:** 11001010

1. Calcola il byte che deve essere scritto sul disco di parità **P** (effettuando lo XOR bit a bit tra $D1, D2, D3$).
2. **Simulazione guasto:** Il disco **D2** si rompe ed è illeggibile. Mostra come il controller RAID può ricostruire i dati di **D2** usando solo $D1, D3$ e P. dimostratelo in maniera formale usando le proprietà dello XOR

b)

1. Calcola il bit di parità usando lo XOR: $1 \oplus 1 \oplus 0 \oplus 1 = ?$
2. Supponiamo che il messaggio ricevuto sia 1001 (il secondo bit è cambiato). La parità calcolata sul messaggio ricevuto corrisponde a quella originale?
3. **Il limite:** Supponiamo che il messaggio ricevuto sia 0001 (il primo e il secondo bit sono cambiati). La parità rileva l'errore? Perché questo rende lo XOR "peggiore" di Hamming?

webserver per esercizi relativi al CPU scheduling:
https://ess.cs.tu-dortmund.de/Software/AnimOS/CPU-Scheduling/
vi permette di inizializzare a random e poi fornisce le soluzioni, essenzialmente sorgente infinita di esercizi di tale tipo.
Guardate inoltre
http://cpuburst.com/

simulatore per esercizi sulla gestione memoria secondaria,
http://www.ecs.umass.edu/ece/koren/architecture/Disk/demo.htm
interfaccia grafica pessima, sono presenti solo 3 algoritmi.
Però sempre webserver per avere esercizi infiniti aggiuntivi a quelli dell'eserciziario.

In https://www.gatevidyalay.com/operating-system/ ci sono molti esempi, ed esercizi spiegati passo per passo, di metodi visti nel corso.
E.g. in https://www.gatevidyalay.com/semaphore-in-os-practice-problems/
Esempi semplici per capire funzionamento semafori

Pagina web per esercizi riguardanti algoritmo del banchiere
https://jangidbhanu.github.io/BankersAlgorithm/

Software per simulare concetti visti a lezione (dunque potete crearvi infiniti esercizi): "Our graphical tool was designed to run on top of a solution to a text-based course programming project—here after referred to as the underlying simulation engine—in which students design and implement a system that simulates some of the job and cpu scheduling, and semaphore processing of a time-shared operating system (os).)
https://cpudemo.azurewebsites.net/#/
leggete il paper relativo per le spiegazioni:
https://arxiv.org/pdf/1812.05160.pdf